



Programa Profesional en Full Stack Developer

Módulo 4. BBDD relacionales

MySQL y no relacionales

MONGO DB

Índice

Tema 1. Qué es una BBDD relacional

- 1.1. Introducción y objetivos
- 1.2. Conceptos de base de datos relacionales
- 1.3. Características
- 1.4. Ventajas y desventajas

Tema 2. Modelo de datos MySQL

- 2.1. Introducción y objetivos
- 2.2. Tipos de datos en MySQL y sus características
- 2.3. ¿Cómo saber qué tipo de datos asigno a un campo dentro de una tabla?
- 2.4. Algunos puntos a tener en cuenta cuando desarrollamos un modelo de datos

Tema 3. Cómo construir una base de datos

- 3.1. Introducción y objetivos
- 3.2. Instalar MySQL dentro de nuestro equipo
- 3.3. Instalar MySQL Workbench para manejo de BBDD
- 3.4. Creación de nuestro primer Modelo de BBDD

Tema 4. Modelo entidad relación

- 4.1. Introducción y objetivos
- 4.2. ¿Qué es el concepto de entidad relacional y cardinalidad?
- 4.3. Crear un modelo ER a través de MySQL Workbench y relacionarlo
- 4.4. Contraints o protección de relaciones

Tema 5. Sentencias CRUD con MySQL

- 5.1. Introducción y objetivos
- 5.2. Creación, vaciado y borrado de tablas
- 5.3. Insertar, borrar y actualizar un registro
- 5.4. Select, la potencia de las consultas dentro un BBDD

Tema 7. Introducción y Diseño de estructuras de BBDD

- 7.1. Introducción y objetivos
- 7.2. Diseño de bases de datos NoSQL
- 7.3. Concepto de documento y colección
- 7.4. Buenas prácticas

Tema 8. Mongo Setup

- 8.1. Introducción y objetivos
- 8.2. Instalación de MongoDB para sistemas Windows
- 8.3. Instalación de MongoDB para sistemas Linux
- 8.4. Instalación de MongoDB para sistemas MacOS

Tema 9. Mongo Shell y JSON

- 9.1. Introducción y objetivos
- 9.2. Configuración inicial de Mongo Shell
- 9.3. Creación de scripts para la Shell
- 9.4. Tipos de datos

Tema 10. CRUD y Query/Update operators

- 10.1. Introducción y objetivos
- 10.2. Concepto de CRUD

10.3. Create Operations

10.4. Read Operations

10.5. Update Operations

10.6. Delete Operations

Tema 1. Qué es una BBDD relacional

1.1. Introducción y objetivos

El objetivo fundamental de este tema es entender cómo funciona una base de datos como herramienta dentro del desarrollo web. Es decir, como podemos crear bases de datos que nos ayuden a tener persistencia de datos dentro de nuestras páginas web.

Dentro de las BBDD una parte muy importante de los desarrollos se realizan a través de BBDD relacionales. En este punto vamos a ver en qué consisten, como se crea un modelo de datos y sobre todo la importancia de relacionar para evitar datos duplicados en nuestras estructuras web.

Entender estos conceptos teóricos de BBDD son fundamentales para comprender como funcionan las bases de datos relacionales en la vida real, y trabajar con ella en el desarrollo de productos web escalables.

Tema 1. Qué es una BBDD relacional

1.2. Conceptos de base de datos relacionales

La principal definición de una base de datos relacional es que permite guardar datos de manera relacionada entre diferentes entidades o tablas. El modelo relacional es una forma sencilla y directa de representar datos sin necesidad de jerarquizarlos.

Las bases de datos relacionales son el modelo más utilizado en la actualidad dentro del desarrollo web.

Una base de datos es un conjunto de tablas o entidades que permiten guardar registros con diferentes columnas o campos. Cada registro tiene un campo que lo hace único denominado ID o clave primaria y el resto de las columnas contiene datos de diferentes tipologías dentro del registro.

Como hemos dicho una de las características principales de una base de datos relacional es evitar la duplicidad de registros y datos a la hora de almacenarlos y así garantizar que la integridad de los datos no se vea afectada en ningún momento.

Además, gracias a las claves primarias y foráneas no solo podemos relacionar registros de tablas si no también acceder a los valores de los campos de forma fácil y rápida.

Una base de datos relacional tiene reglas que evita que cometamos errores a la hora de crearlas. Por ejemplo:

- ▶ No puede existir dos tablas con el mismo nombre.
- ▶ La relación entre tablas se realiza a través de las claves primaria y foránea de cada tabla.

Tema 1. Qué es una BBDD relacional

1.3. Características

Una de las principales características de las bases de datos relacionales es que evitan la duplicidad de datos dentro de los registros generando referencias entre las tablas y protegiendo la integridad de estos, es decir que si se elimina uno de los registros el resto no se verá afectado.

El **uso de las claves** es vital para este tipo de bases de datos ya que nos aporta una forma fácil de acceder a la información y recuperarla.

Para poder gestionar una base de datos relacional necesitamos un sistema de gestión de bases de datos relacionales. La principal tarea de este sistema es el permitir gestionar de forma fácil, el almacenamiento y rendimiento de una base de datos relacional. Este sistema se escoge en función de las necesidades del desarrollo, algunos tienen un coste de licencia y otros son OpenSource como en el caso de MySQL.

Cuando generamos una base de datos relacional casi siempre separamos el esquema de los datos, es decir definimos inicialmente una estructura y luego posteriormente la llenamos de datos.

Es más, la misma estructura puede almacenar distintos datos siguiendo el mismo modelo, por ejemplo, un hospital, mientras la estructura con los campos sea la misma yo puedo usar mi tabla para almacenar datos de los diferentes hospitales.

Tema 1. Qué es una BBDD relacional

1.4. Ventajas y desventajas

Como cualquier sistema no todo es perfecto y el uso de este tipo de bases de datos tiene sus ventajas y desventajas. En este punto creo que existen más cantidad de ventajas que desventajas por eso hace que sea una de las bases de datos más usadas.

Ventajas

a **principal ventaja** de este modelo de base de datos radica en la **sencillez** con la que nos permite manejar gran cantidad de datos y que estos estén relacionados unos con otros, haciendo que su gestión sea segura y en base a unas normas.

Las **bases de datos relacionales** me permiten hacer copias de estas de forma fácil sabiendo que dichas copias son exactamente iguales y se producen de forma inmediata.

Como ya hemos comentado **antes evitan la duplicidad de registros** dentro de la misma base de datos.

Este tipo de base de datos **permite hacer peticiones simultaneas de varios usuarios** a la vez.

Desventajas

Una de las principales desventajas de las bases de datos relacionales es que **no permiten desarrollar tablas organizadas de forma jerárquica**, es decir, que no se puede crear una subfila dentro de otra fila. Es decir, no se pueden crear tablas subordinadas.

Tema 1. Qué es una BBDD relacional

El **acceso a los datos** es **lento**, ya que, para darnos un resultado, muchas veces tenemos que consultar a más de una tabla y ese proceso es más lento que si tuviéramos que consultar los datos a una sola.

En la actualidad existen varios tipos de gestores de bases de datos relacionales, los más comunes son:

- ▶ Oraclezz
- ▶ MySQL.
- ▶ Microsoft SQL server.
- ▶ PostgreSQL.
- ▶ DB2.

En este curso nosotros vamos a trabajar con MySQL o su fork Maria DB que manejan lenguaje SQL y están catalogadas como software libre, luego no tenemos que pagar ningún tipo de licencia para usarla.

Tema 2. Modelo de datos MySQL

2.1. Introducción y objetivos

El objetivo principal de este tema es poner sobre la mesa los diferentes tipos de datos con los que trabaja MySQL y con lo que podemos generar lo que se llama un modelo de datos. Para ello vamos a estudiar de manera muy concreta los diferentes tipos de datos y el espacio que ocupan, esto nos dará herramientas para hacer nuestras BBDD más rápidas y menos pesadas.

Debemos acostumbrarnos a que cuando creemos una BBDD debemos estudiar qué tipo de dato nos conviene ya que la decisión entre elegir un tipo de dato numérico o de cadena puede influir ligeramente en el rendimiento de nuestra BBDD, si sumamos todos los campos y todas las tablas de nuestra BBDD relacional al final podemos conseguir una BBDD más lenta por no pensar en un buen modelo de datos.

Este tema es más un concepto teórico pero muy importante a la hora de manejar una BBDD. Manejar bien los tipos de datos es el principio de un desarrollo óptimo de una BBDD relacional.

Tema 2. Modelo de datos MySQL

2.2. Tipos de datos en MySQL y sus características

Antes de empezar a crear campos con tipos de datos vamos a repasar bien las partes de una BBDD relacional.

En primer lugar, tenemos una tabla, es el conjunto de información que tiene relación entre sí, es decir productos, usuarios, etc.

Un conjunto de tablas forma lo que se conoce como base de datos, es una entidad de información mayor que en muchos sitios la verás definida como schema.

Las tablas pueden tener relación entre sí y ser lo que se denomina como complementarias, o pueden no tener ningún tipo de relación.

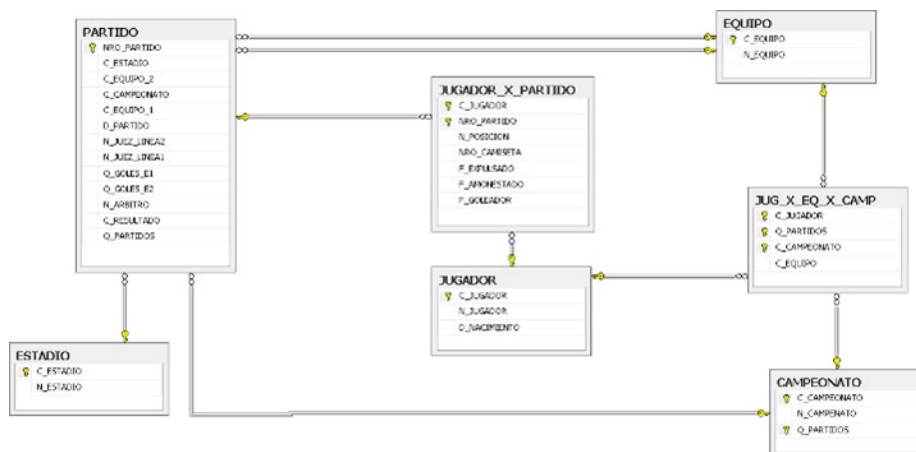


Figura 1. Diagrama de juego de Ajedrez. Muestra de un modelo de datos. Fuente:

<https://bbddupc.wordpress.com>

Una tabla a su vez está formada por registros y cada registro está formado por campos o columnas, que son la unidad básica de información, y son una el valor concreto de un objeto o registro. Cada registro debe tener una serie de campos que hacen específico ese registro.

Cada campo o columna tendrá asociado un tipo de dato específico, que te indica de

Tema 2. Modelo de datos MySQL

alguna forma lo que se va a almacenar dentro de él, sus características y sus limitaciones.

¿Cómo se diseña una BBDD?

continuación, vamos a describir una serie de puntos que creemos que si los sigues más o menos de forma fiel te ayudarán en el proceso de creación de una BBDD relacional con MySQL.

- ▶ Cada tabla debe de tener un nombre único, descriptivo y en plural, por ejemplo, usuarios, productos.
- ▶ Cada una de las tablas debe tener como mínimo un campo.
- ▶ Las tablas no tienen obligación de tener datos pueden estar vacías. Aunque pueden tener n filas y están pueden estar desordenadas.
- ▶ Cada valor en un campo de un registro tendrá el mismo tipo de dato que el otro registro de la misma tabla.
- ▶ Debe existir al menos un campo que identifique de forma inequívoca dicho registro, a este campo se le conoce como **llave primaria**.
- ▶ Solo puede existir **una llave primaria por tabla**, es decir dentro de una tabla solo puede haber un campo que sea clave primaria.
- ▶ Una **llave secundaria (clave foránea o foreign key)** es un campo que es **llave primaria** en otra tabla.
- ▶ La relación que existe entre las **llaves primaria y foranea**, marca un relación padre e hijo entre dos tablas de unas bases de datos que se conoce como entidad relación y que veremos en un tema más adelante.

Una vez que tenemos claros los paso para crear una base de datos vamos a definir

Tema 2. Modelo de datos MySQL

que tipos de datos podemos usar dentro de los campos de una tabla. MySQL hace esta distinción.

Numéricos:

Los números con valores muy importante dentro de una BBDD pueden ser enteros, racionales o en coma flotante y se pueden guardar como decimal o hexadecimal.

MySQL reconoce las notaciones científicas como por ejemplo el numero e o el logaritmo.

Cadenas de caracteres:

Las cadenas de caracteres son lo que se conoce como texto en cualquier lenguaje de programación, «Esto es un texto o cadena». Al igual que en otros lenguajes de programación pueden usarse comillas dobles o sencillas para agrupar la cadena de caracteres.

Fecha y hora:

Fechas y horas son valores del tipo «1999-06-17» o «12:30:43». También reconoce las combinaciones de ambos valores como «1999-06-17 12:30:43».

MySQL tiene un formato de fecha de YYYY-MM-DD HH:MM:SS, sin embargo a través de la función DATE_FORMAT() podemos cambiar dicho formato.

Los formatos fecha y hora en MySQL son un subconjunto de las cadenas de caracteres ya que, aunque son un tipo de dato a parte mantienen la forma de trabajar de las cadenas de caracteres con el uso de comillas.

Tema 2. Modelo de datos MySQL

NULL:

Se puede considerar un valor «sin tipo». Normalmente quiere decir «sin valor», «valor desconocido», etc. Puede insertar en la tabla valores NULL y hacer selecciones de campos si tienen en valor NULL, etc.

Se usa para poder vaciar de valor y de tipo un campo específico.

No se puede realizar ningún tipo de operación con el valor NULL, pues el resultado seguirá siendo NULL.

En MySQL usaremos NULL para romper la referencia entre dos tablas, por ejemplo.

Tema 2. Modelo de datos MySQL

2.3. ¿Cómo saber qué tipo de datos asigno a un campo dentro de una tabla?

Cada vez que queremos asignar un tipo de dato a una tabla no solo debemos conocer el listado de tipo y sus características, si no pensar para que los vamos a usar dentro de nuestra BBDD. Para ello debemos hacernos las siguientes preguntas.

- ▶ ¿Qué clase de valores voy a almacenar en dicho campo?
- ▶ ¿Cuánto espacio ocupan los valores, y si son de longitud fija o de longitud variables?
- ▶ ¿Cómo se clasifican o comparan los valores del tipo??
- ▶ Si el tipo permite o no valores NULL.
- ▶ Si el tipo que hemos creado se puede indexar de alguna forma o no.

Hacer un buen estudio de los tipos de los campos es muy importante de cara a reducir el tamaño de la base de datos y aumentar su velocidad en el desarrollo de los procesos.

Para ello es importante conocer bien los tipos y para que se usan.

Empecemos con los de tipo entero.

- ▶ **TINYINT**: Es un entero muy pequeño apenas requiere un byte. Su rango de valores va desde el -128 a 127 y sin signo va desde 0 al 255 ya que toda la parte negativa se vuelca hacia el lado positivo.
- ▶ **MEDIUMINT**: Es un entero mediano que requiere 3 byte de peso. Con signo va desde -8388608 a 8388607 y sin signo 0 a 1677215.
- ▶ **INT**: Es un entero mediano, aunque se le puede designar el tamaño cuando se crea. Requiere 4 bytes y su rango de valores va desde -2147683648 a 2147483647 y sin

Tema 2. Modelo de datos MySQL

signo 0 a 4294967295.

- ▶ **BIGINT**: Es un entero muy grande que requiere 8 bytes de peso. Con signo va desde -9223372036854775808 a 9223372036854775807 y sin signo 0 a 18446744073709551515

El rango de los cinco tipos de enteros depende de si tiene o no signo (unsigned), que es uno de los atributos.

Un atributo es una característica adicional a la columna.

Para seleccionar algún tipo de columna necesitamos analizar el rango de nuestra información y seleccionar el tipo más pequeño que satisfaga ese rango, de lo contrario estaremos desperdiciando recursos de nuestra base de datos.

Si usted intenta almacenar un campo mayor al rango del tipo de columna, el proceso marcará un error y sólo cargará hasta el rango máximo, por ejemplo:

Tiene un campo tipo TINYINT con signo, y almacena un valor como 500, solo se almacenará 127.

Veamos más atributos de campos numéricos.

- ▶ **ZEROFILL**: Tanto en Enteros como en punto flotante, se tiene el atributo. ZEROFILL que sirve para rellenar la parte izquierda de la columna en cero.
- ▶ **AUTO_INCREMENT**: El sistema numerará desde el número uno a cada uno de los renglones que sean creados en la tabla. Solo se puede tener un campo de este tipo y debe ser nombrado como una llave (PRIMARY KEY) o como clave (UNIQUE). No acepta valores nulos (NOT NULL).
- ▶ **UNSIGNED**: Este comportamiento rechaza los valores negativos de una columna y se ajusta el rango aceptado como se muestra en la tabla de los números enteros.

Vamos a ver ahora los de tipo decimal.

Tema 2. Modelo de datos MySQL

- ▶ **FLOAT:** También denominado número en coma flotante requiere 4 bytes de peso y su rango es muy alto va desde $-1.175494351E-38$.
- ▶ **DECIMAL (M, D):** es un numero en coma flotante representado como una cadena o string, requiere M+2 bytes y depende directamente del valor de M y D.

Los números de coma flotante siempre son positivos y negativos. No tenemos atributos UNSIGNED.

- ▶ DECIMAL(4,1) -99.9 a 999.9
- ▶ DECIMAL(5,1) -999.9 a 9999.9
- ▶ DECIMAL(6,1) -9999.9 a 99999.9
- ▶ DECIMAL(6,2) -999.99 a 9999.99
- ▶ DECIMAL(6,3) -99.999 a 999.999

MySQL proporciona varios tipos de cadena para mantener los datos de un carácter. Para todos los tipos de cadena de caracteres, los valores que son demasiados largos se acortan para ajustarlos.

Las cadenas son valores como «MySQL es una base de datos» o «la vida puede ser maravillosa».

Puede utilizar comillas dobles o sencillas para agrupar a una cadena.

- ▶ **CHAR** Cadena de caracteres de longitud fija.
- ▶ **VARCHAR** Cadena de caracteres de longitud variable.
- ▶ **TINYBLOB** Binary Large Object (BLOB) muy pequeño.
- ▶ **BLOBBLOB** pequeño.

Tema 2. Modelo de datos MySQL

- ▶ **MEDIUMBLOB** Mediano.
- ▶ **LONGBLOB** Largo.
- ▶ **TINYTEXT**: Cadena de texto muy pequeña.
- ▶ **TEXT**: Cadena de texto de longitud pequeña.
- ▶ **MEDIUMTEXT**: Cadena de texto de longitud mediana.
- ▶ **LONGTEXT**: Cadena de texto larga.
- ▶ **ENUM**: Una enumeración: columnas a las que se pueden asignar un miembro de enumeración.
- ▶ **SET**: Un conjunto: columnas a las que se pueden asignar múltiples conjuntos de miembros.

Los tipos de datos de tipo cadena más usados son el **CHAR Y VARCHAR**, el primero es de longitud fija y el segundo como su propio nombre indica de longitud variable. Si los valores que vas a almacenar en los campos no varían de longitud muy descaradamente (por ejemplo, un número de teléfono) la elección sería el uso de **CHAR**, pues **VARCHAR** posee un byte adicional que sirve de control, con lo que los campos de longitud fija se procesan más rápido.

Si el campo es de longitud variable todos los registros que almacenen un dato serán de longitud variable, como por ejemplo un nombre que puede ser «Ana» o «Juan Antonio». Si nuestra **BBDD** tiene un problema de espacio lo mejor será usar un tipo de datos de longitud variable. Si combinas columnas **CHAR** y **VARCHAR**, **MySQL** convertirá las columnas **CHAR** (con una longitud mayor a 4) a tipo **VARCHAR**.

MySQL tiene un bloque de campos **TEXT (TINYTEXT, TEXT, MEDIUMTEXT, LONGTEXT)** con longitud variable de los textos. También sirve para almacenar texto enriquecido en formato HTML.

Tema 2. Modelo de datos MySQL

Por último, le vamos a dar un repaso a los textos en formato fecha porque las fechas son elementos muy importantes dentro del desarrollo de base de datos ya que nos permite guardar cuando ocurren los registros y cuando se actualizan, a parte los elementos como fechas de nacimiento, etc.

MySQL proporciona varios tipos de columna para valores temporales.

- ▶ **DATE:** Requiere 3 bytes. Valor de fecha en formato YYYY-MMDD "1000-01-01" a "9999-12-31"
- ▶ **TIME:** Valor de hora en formato HH:MM:SS "-838:59:59" a "838:59:59"
- ▶ **DATETIME:** Valor de fecha y hora en formato YYYY-MM-DD hh:mm:ss "1000-01-01 00:00:00" a "9999-12-31 23:59:59" **TIMESTAMP :** Valor de un lapso de tiempo en formato YYYYMMDDhhmmss
- ▶ **YEAR** Año en formato YYYY.

Tema 2. Modelo de datos MySQL

2.4. Algunos puntos a tener en cuenta cuando desarrollamos un modelo de datos

Para poder escoger entre todas las variaciones de tipo de columna, no está demás hacerse las siguientes preguntas (aunque algunas veces parecerán obvias):

- ▶ ¿Qué tipo de valores, números, cadenas y fechas almacenará la columna?
- ▶ ¿Sus valores se corresponden con algún rango?
- ▶ ¿Qué tipos de columnas son los más eficientes?
- ▶ Operaciones de cadena vs. numéricas.
- ▶ Tipos más pequeños vs. tipos más grandes.
- ▶ Tipo de longitud fija vs. longitud variable.

Tema 3. Cómo construir una base de datos

3.1. Introducción y objetivos

El objetivo de este tema es entender cómo se maneja MySQL y como se crea un BBDD. Para ello vamos a usar una herramienta muy útil que es un gestor de BBDD, en este caso el oficial de la propia base de datos, que es MySQL Workbench.

Durante este tema vamos a ver y entender cómo se instala en nuestros equipos MySQL y las herramientas necesarias para trabajar con ella.

Vamos a entender también que es un servicio de base de datos y como se activa para poder activarlo y desactivarlo.

Y por último vamos a ver cómo podemos crear una base de datos a través de estas herramientas de forma sencilla, ya que MySQL Workbench me facilita muchísimo la labor en este aspecto.

Tema 3. Cómo construir una base de datos

3.2. Instalar MySQL dentro de nuestro equipo

Vamos a comenzar por la instalación de MySQL, en nuestro equipo, el proceso que te muestro es el de instalación en Windows, pero puedes encontrar en el apartado de a fondo, más información sobre la instalación en diferentes sistemas operativos.

Lo primero para instalar MySQL es ir a la página oficial de la BBDD en: <https://dev.mysql.com/downloads/installer/>

Desde ahí puedes descargar la última versión para tu sistema operativo, elige siempre la que pese más porque la otra es solo un instalador que luego se conecta a internet y descarga lo que necesita. Es más cómodo y rápido el enlace más pesado.

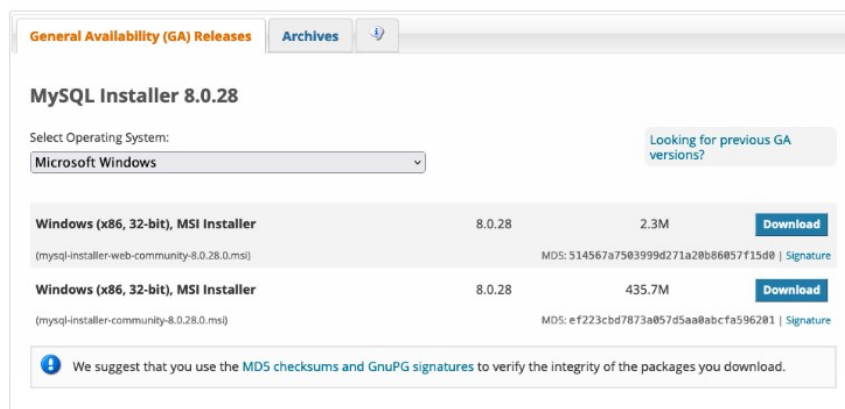


Figura 2. Elección de descargable. Fuente: elaboración propia.

Te pedirá que te registres, pero no hace falta, ya puedes saltarte ese paso haciendo click en el enlace de abajo.

Tema 3. Cómo construir una base de datos

MySQL Community Downloads

Login Now or Sign Up for a free account.

An Oracle Web Account provides you with the following advantages:

- Fast access to MySQL software downloads
- Download technical White Papers and Presentations
- Post messages in the MySQL Discussion Forums
- Report and track bugs in the MySQL bug system

Login »
using my Oracle Web account

Sign Up »
for an Oracle Web account

MySQL.com is using Oracle SSO for authentication. If you already have an Oracle Web account, click the Login link. Otherwise, you can sign up for a free account by clicking the Sign Up link and following the instructions.

[No thanks, just start my download.](#)

Figura 3. Proceso de descarga. Fuente: elaboración propia.

Una vez descargado accionas el instalador y vamos a configurar nuestra instalación de MySQL para que no haya ningún problema y podamos trabajar con ella.

Vamos a intentar mostrar los pasos poco a poco.

Tema 3. Cómo construir una base de datos

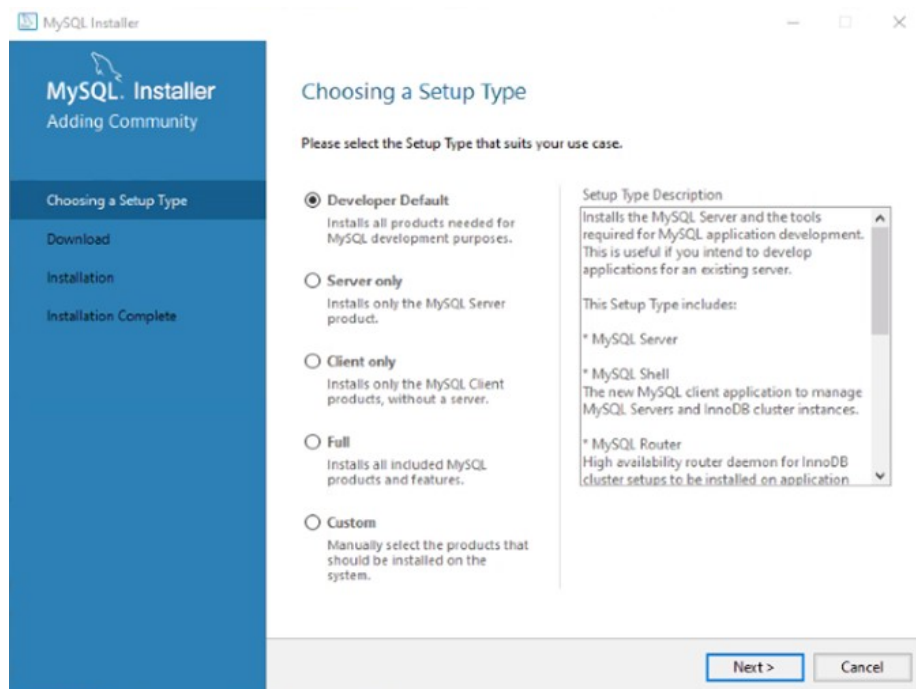


Figura 4. Proceso de instalación 1. Fuente: elaboración propia.

En la primera pantalla debes elegir la opción Custom, que te permitirá elegir los elementos necesarios para usar MySQL como desarrollador y no instalar todo ya que no es necesario.

Tema 3. Cómo construir una base de datos

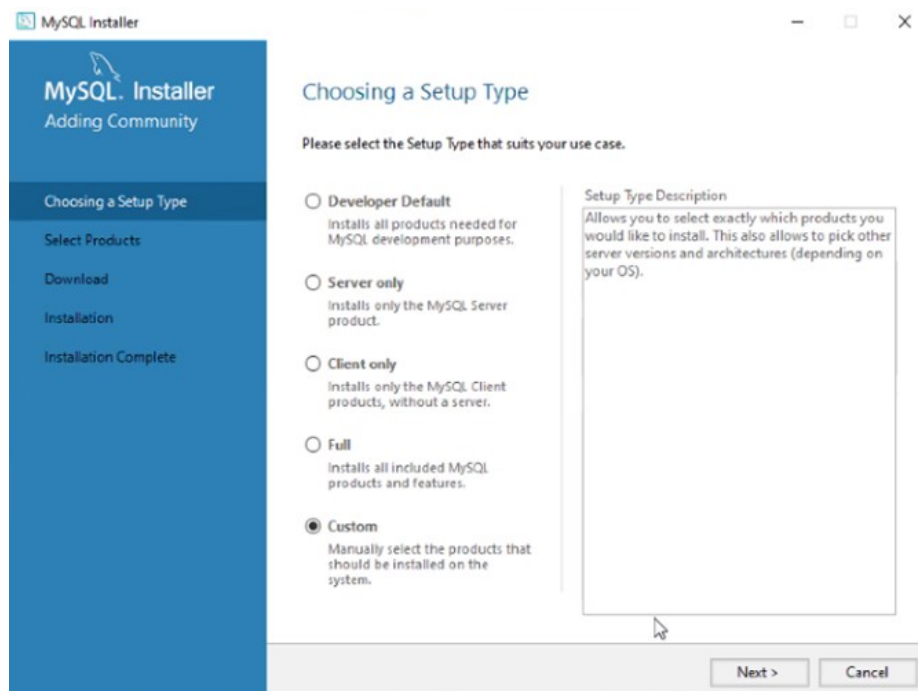


Figura 5. Proceso de instalación 2. Fuente: elaboración propia.

Una vez que le demos a siguiente empezaremos a elegir cada uno de los elementos que necesitamos instalar.

Como puedes ver en la captura de pantalla solo necesitamos MySQL Server en su última versión y MySQL WorkBench que es programa que necesitamos para manejar nuestras BBDD.

Tema 3. Cómo construir una base de datos

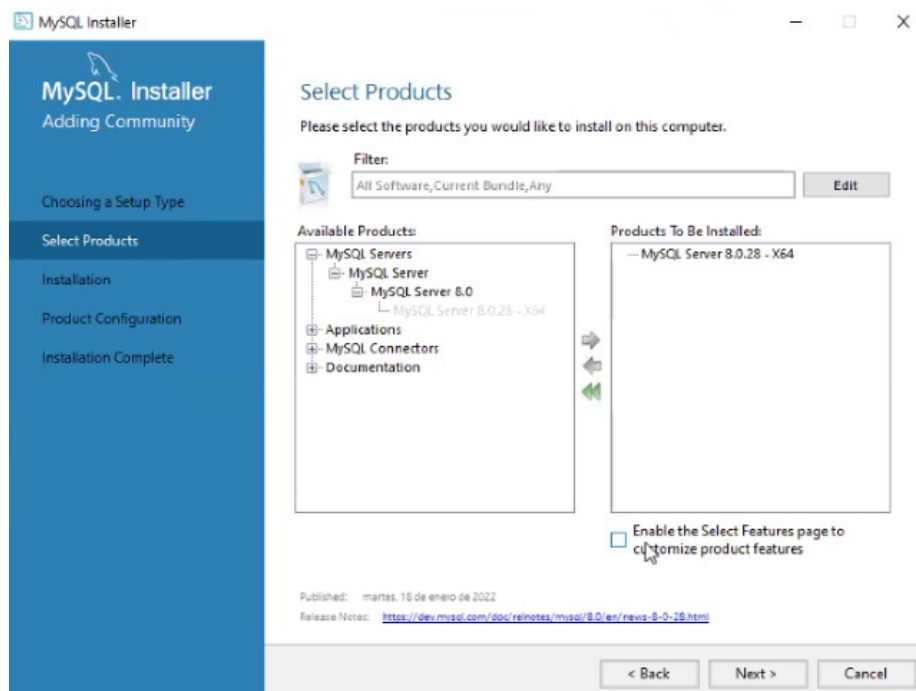


Figura 6. Proceso de instalación 3. Fuente: elaboración propia.

Tema 3. Cómo construir una base de datos

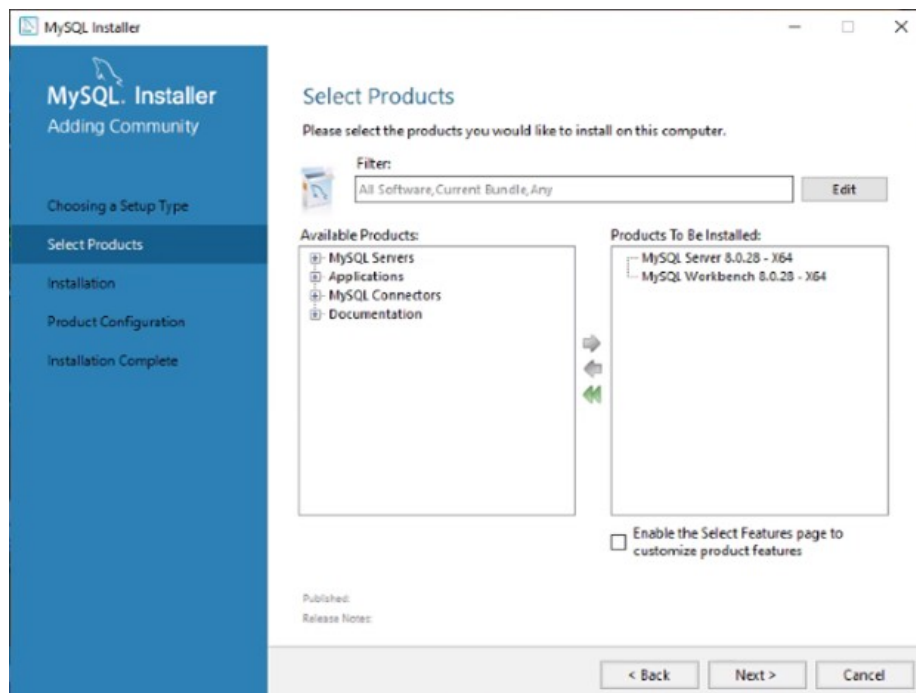


Figura 7. Proceso de instalación 4. Fuente: elaboración propia.

Si todo ha ido correctamente y no te pide instalar ninguna librería externa, que a veces ocurre, depende de la versión del sistema operativo. Te saldrá una pantalla donde podrás comenzar la instalación de todo el proceso.

Tema 3. Cómo construir una base de datos

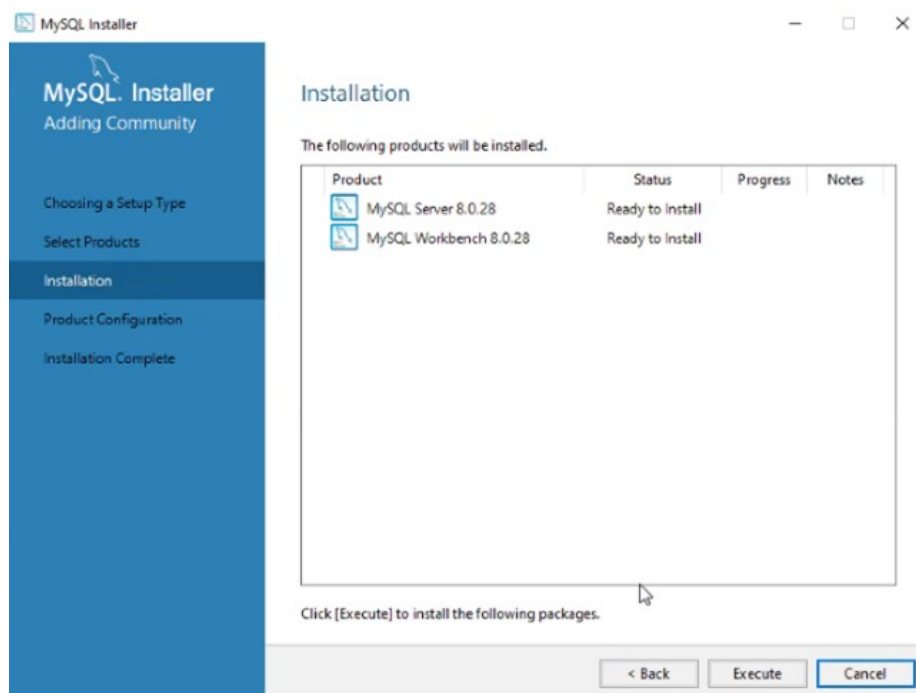
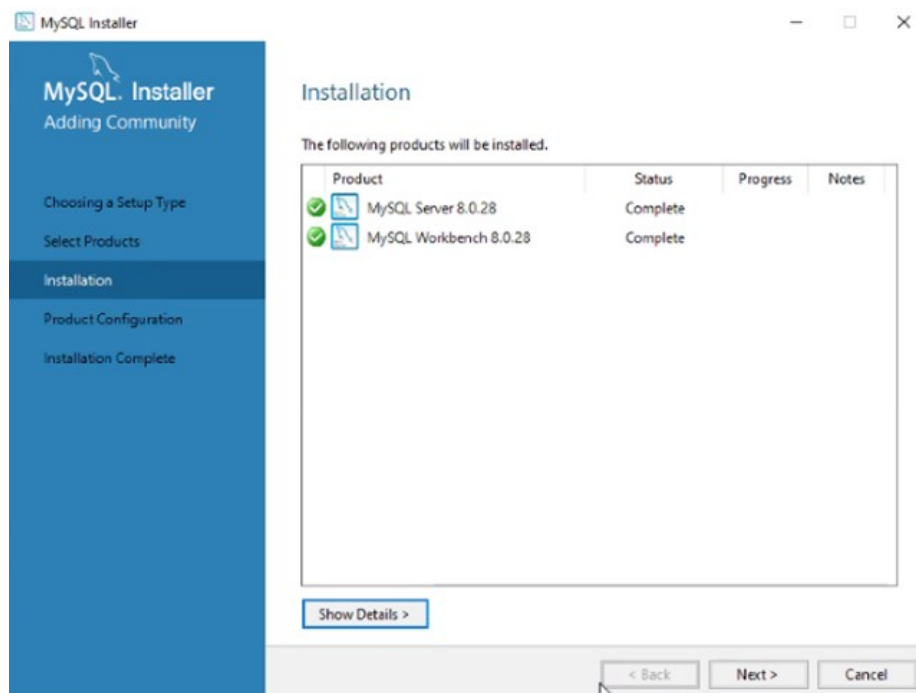


Figura 8. Proceso de instalación 5. Fuente: elaboración propia.

Lo ejecutamos y esperamos a que se complete.



Tema 3. Cómo construir una base de datos

Figura 9. Proceso de instalación 5. Fuente: elaboración propia.

Una vez que hayamos terminado la instalación llega el proceso más importante que es la configuración de nuestra base de datos. Vamos a ir revisando cada uno de los pasos para que facilitarte la labor a la hora de instalar.

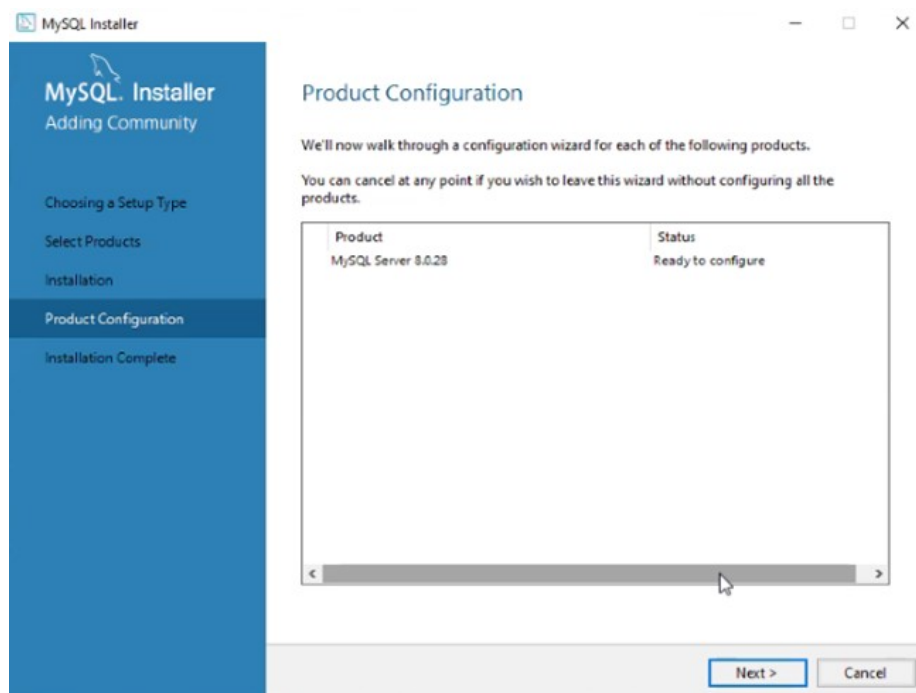


Figura 10. Proceso de configuración 1. Fuente: elaboración propia.

Tema 3. Cómo construir una base de datos

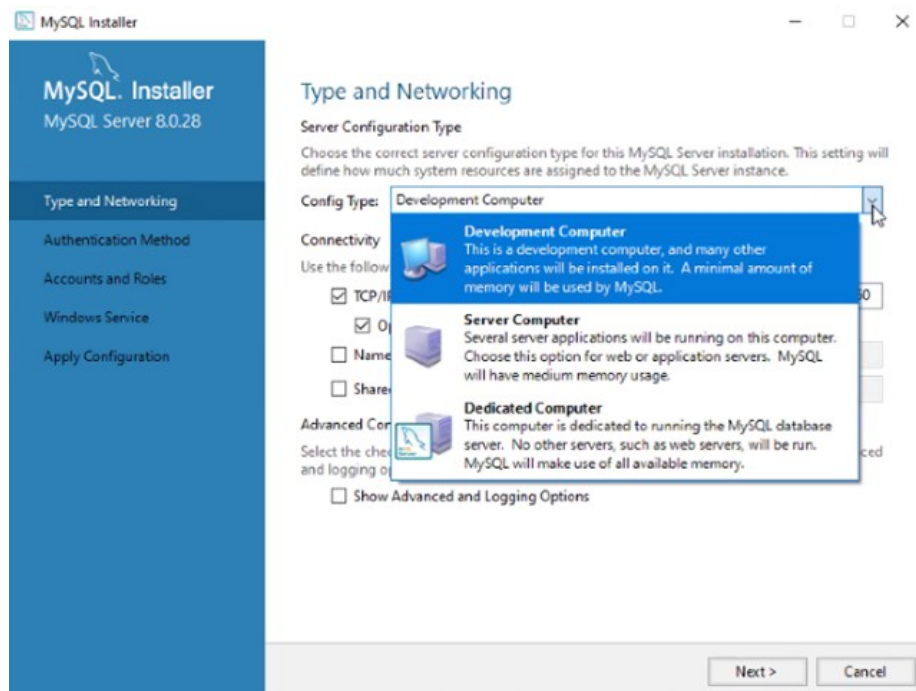
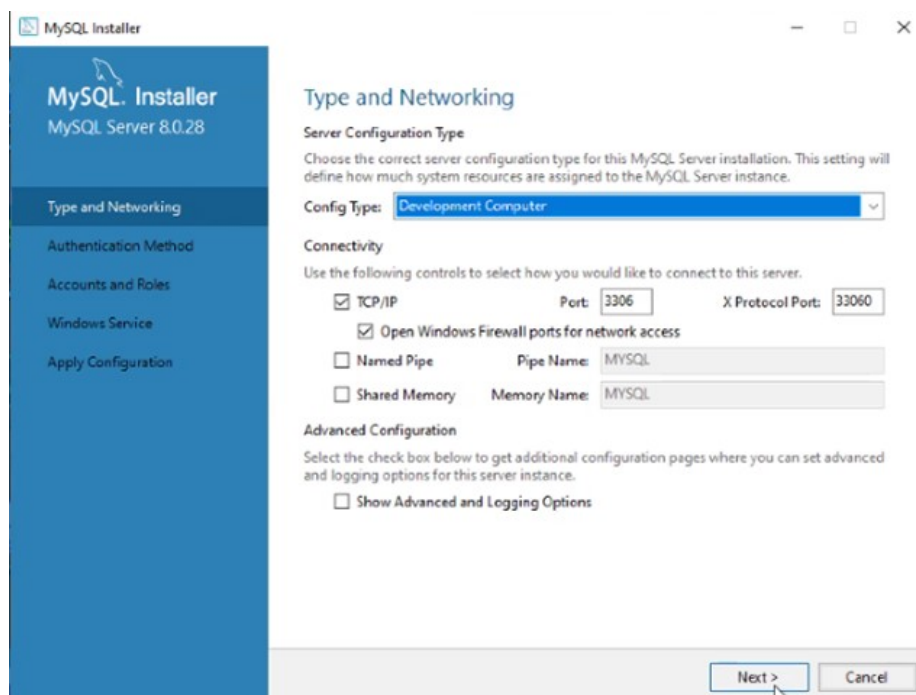


Figura 11. Proceso de configuración 2. Fuente: elaboración propia.

La primera configuración a escoger es que nuestro equipo es un equipo de desarrollo con lo que las configuraciones que vamos a hacer es Development Computer.



Tema 3. Cómo construir una base de datos

Figura 12. Proceso de configuración 3. Fuente: elaboración propia.

El puerto por defecto de nuestra BBDD es el 3306 y debemos decirle que nos abra el firewall de Windows para que el sistema operativo no bloquee el acceso a ciertas configuraciones.

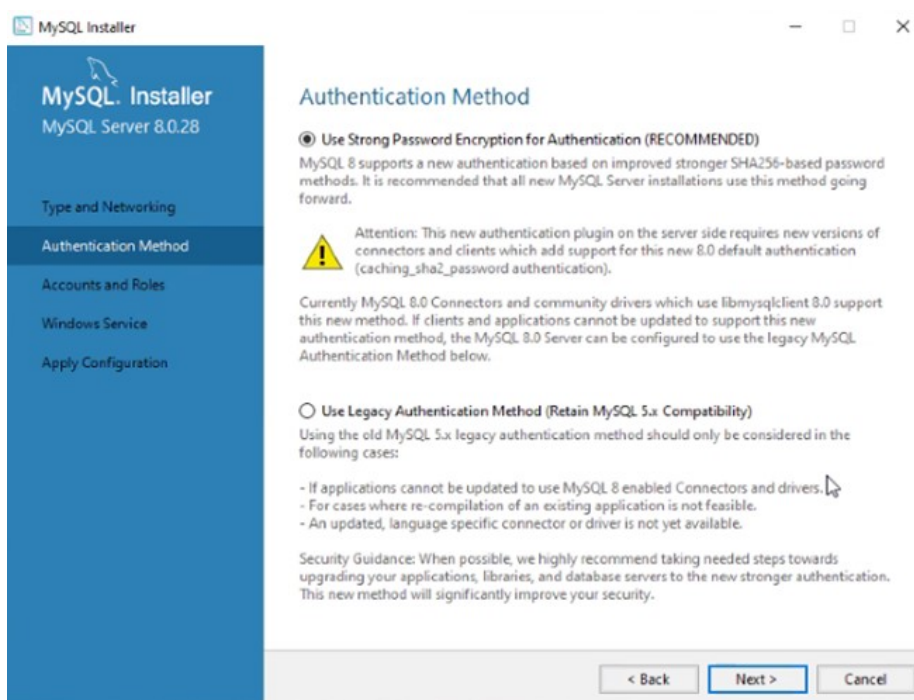


Figura 13. Proceso de configuración 4. Fuente: elaboración propia.

Una vez que pases esa pantalla te pedirá que crees una contraseña para usarás para acceder a MySQL tanto de nivel de código como a nivel de aplicación de MySQL Workbench así que esta contraseña que pongas apuntala en algún sitio ya que la vas a necesitar muchas veces.

Tema 3. Cómo construir una base de datos

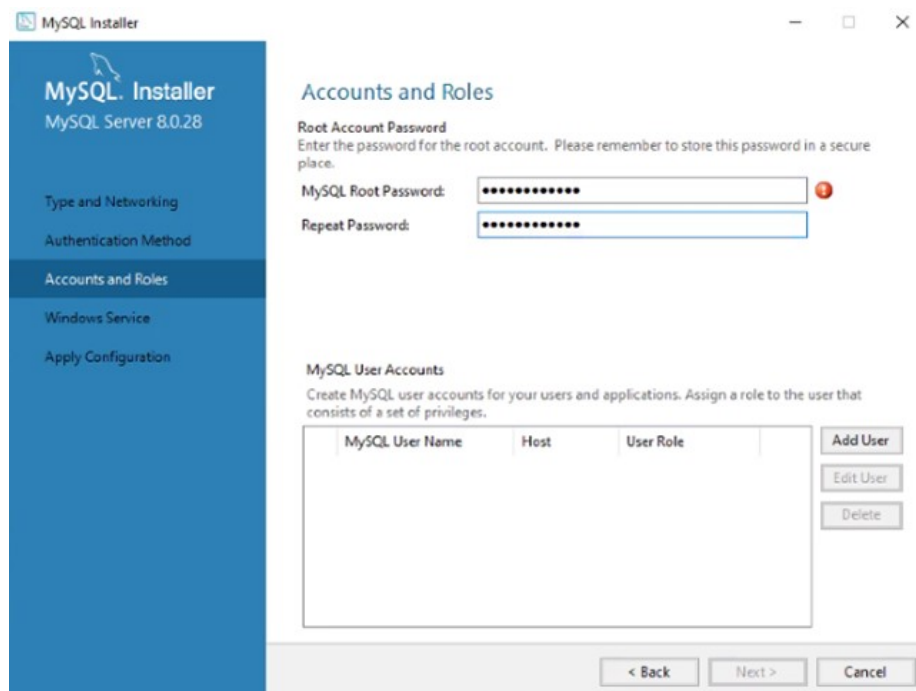


Figura 14. Proceso de configuración 4. Fuente: elaboración propia.

Puedes crear también y usuario, aunque no es estrictamente necesario, ya que si no lo creas si le asigna el usuario por defecto que es root. Debes guardar el usuario y la contraseña, ya que la vas a necesitar a menudo.

Tema 3. Cómo construir una base de datos

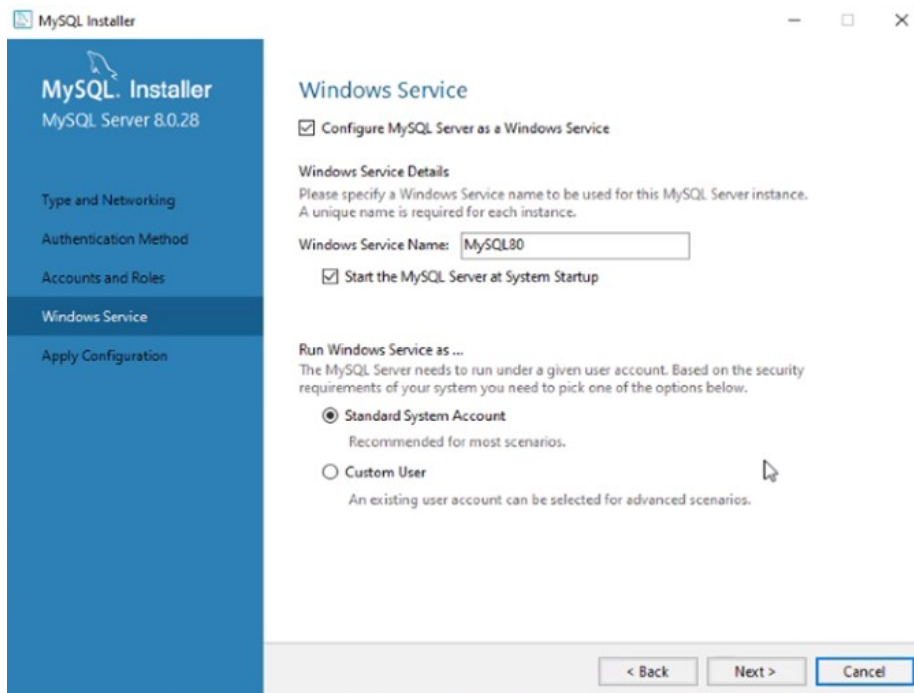


Figura 15. Proceso de configuración 5. Fuente: elaboración propia.

En esta pantalla se configura el servicio de MySQL, básicamente es el nombre que recibe el servicio y le asignamos la posibilidad de que arranque con el sistema operativo, cosa que te recomiendo porque así no te tienes que andar preocupando de arrancarlo cada vez que enciendas el ordenador. Un servicio en este caso es como una tarea que se inicia y se puede parar en cualquier momento y te permite tener la BBDD disponible para tu uso en tu ordenador de forma local sin necesidad de tener un servidor propio en tu equipo. Es muy útil para los desarrolladores.

Una vez acabado este paso los siguientes son simplemente pasos que debes realizar para que se termine la configuración y quede registrada no hay nada más que tengas que hacer, simplemente acabar el proceso dando a next.

Un punto importante es ver dónde está el servicio levantado por si en algún momento quieres o necesitas pararlo o si por la razón que sea se ha parado y no puedes conectar con la base de datos.

Tema 3. Cómo construir una base de datos

Dentro del buscador del sistema operativo vas a buscar la palabra servicios que es una aplicación propia del sistema.

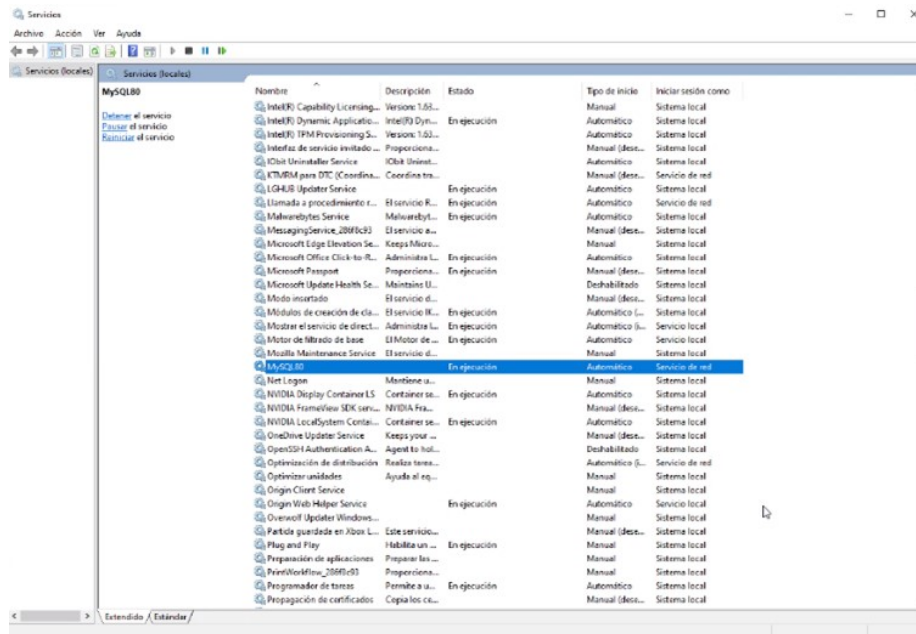


Figura 16. Situar el servicio para poderlo gestionar. Fuente: elaboración propia.

Una vez dentro de dicha aplicación debes buscar el servicio tal y como lo llamaste en la instalación y configuración de MySQL, veras que probablemente estará en ejecución, si das doble click sobre ese servicio verás la siguiente pantalla.

Tema 3. Cómo construir una base de datos

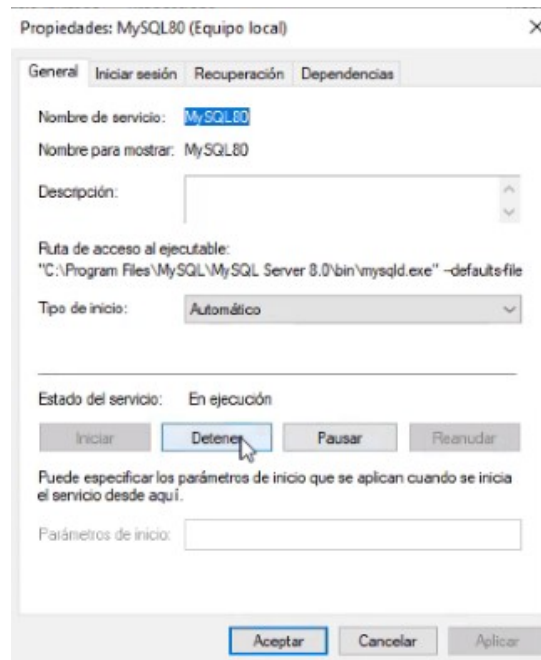


Figura 17. Encendido y apagado del servicio. Fuente: elaboración propia.

Esta pantalla te permitirá apagar y encender el servicio de MySQL a placer.

El proceso de instalación en Mac no es tan largo y complejo, pero por el contrario deberás descargar e instalar MySQL Workbench por separado.

Tendrás que estar atento igualmente al momento en el cual te pedirá crear una contraseña de BBDD durante el proceso.

Tema 3. Cómo construir una base de datos

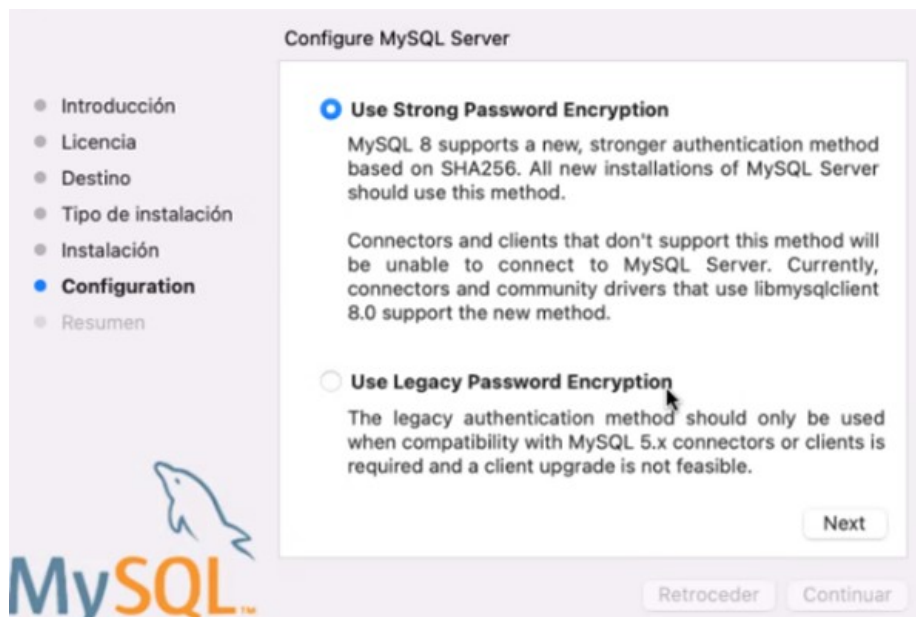


Figura 18. Momento de la petición de contraseña en la configuración de BBDD. Fuente: elaboración propia.

Para poder apagar y encender el servicio, la instalación de MySQL te genera un acceso a MySQL desde las preferencias del sistema, para que puedas apagar y encender el servicio de MySQL.

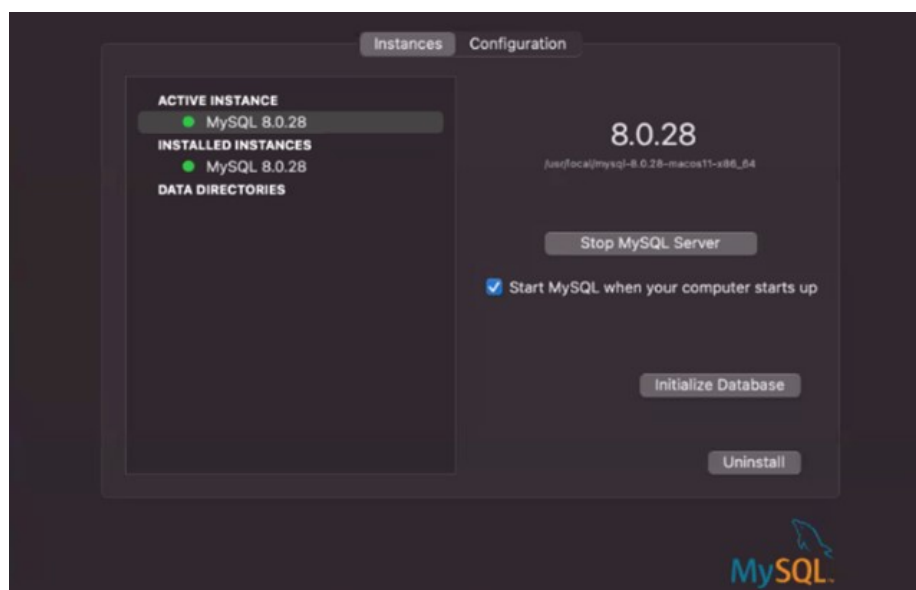


Figura 19. Aplicación de MySQL dentro del preferencias del sistema. Fuente: elaboración propia.

Tema 3. Cómo construir una base de datos

En Linux la cosa cambia ya que tendrás que tener conocimientos avanzados del sistema operativo para instalar las dependencias tanto de MySQL como del Workbench.

También puedes instalar MySQL a través de instaladores tipo LAMP que aparte de MySQL te instalan y configuran otras herramientas como PHP, o Apache. Algunos de los más famosos son.

XAMMP <https://www.apachefriends.org/es/index.html>

MAMP <https://www.mamp.info/en/windows/>

Tema 3. Cómo construir una base de datos

3.3. Instalar MySQL Workbench para manejo de BBDD

La instalación de Workbench en Windows viene adjunta con el proceso de instalación de MySQL, y en Mac simplemente tienes que descargar el instalador y arrastrarlo de forma muy sencilla a la carpeta de aplicaciones.

En Linux el proceso tampoco es muy completo ya que Workbench está disponible en cualquier gestor de paquetes de Linux para su instalación

En interfaz de MySQL Workbench es muy visual y me permite interactuar con mis bases de datos, con las tablas y con los campos de estas.

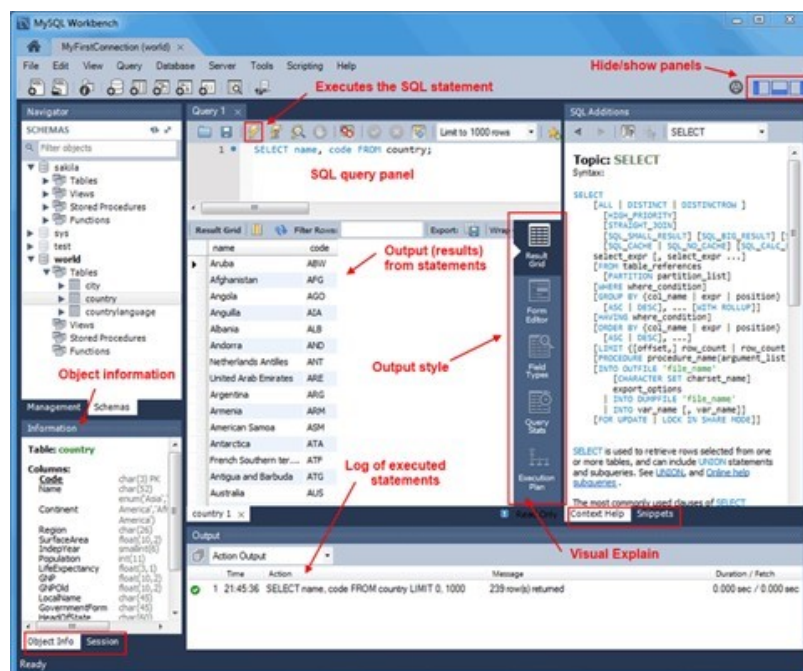


Figura 20. Pantallazo de interfaz MySQL Workbench. Fuente:

https://docs.oracle.com/cd/E17952_01/workbench-en/wb-sql-editor-query-panel.html

Tema 3. Cómo construir una base de datos

La herramienta está muy pensada para la creación de BBDD tanto de forma visual como a partir de código SQL pero principalmente destaca la zona de SCHEMAS que es donde van a estar todas nuestras BBDD, con sus tablas y desde donde podremos acceder a ellas.

Una vez crees una BBDD podrás crear las correspondientes tablas y campos dentro de la misma, donde podremos asignar tipos de datos a cada uno de los campos a golpe de ratón, esto facilita bastante el trabajo ya que te permite tener herramienta de selección y de toma de decisiones sin tener que memorizar cada uno de los valores para ponerlo por código.

Un resumen de pasos para crear una BBDD en MySQL Workbench podría ser el siguiente:

- ▶ Dentro de la zona de SCHEMAS, con el botón derecho del ratón, puedes crear un nuevo SCHEMA.
- ▶ Tendrás que asignarle un COLLATION y un CHARSET a la base de datos, aquí te recomiendo que siempre escojas **utf8mb4_unicode_ci**, ya que tiene disponibles todos los caracteres tanto de la lengua castellana como del resto de caracteres de otras lenguas como la comilla francesa o la cedilla catalana.
- ▶ Una vez que tengas la BBDD creada con el botón derecho del ratón sobre el nombre podrás crear la tablas. Recuerda que las tablas es un conjunto de registro luego los nombres tienen que ser en plural.
- ▶ Haciendo doble click en la tabla se te abrirá en el centro de la aplicación una zona para dar de alta campos dentro de la tabla donde podrás asociar a cada campo su tipo de dato y longitud, así como los atributos de cada campo.
- ▶ A la derecha del nombre de la tabla tienes tres símbolos, dentro de la zona de esquemas, una es una i de información que muestra información sobre la tabla, otro es una llave inglesa que me permite gestionar y crear campos, y otro es un símbolo

Tema 3. Cómo construir una base de datos

de una tabla, que me permite introducir valores dentro de la BBDD, ya sea en formato tabla o a través de un formulario.

Tema 3. Cómo construir una base de datos

3.4. Creación de nuestro primer Modelo de BBDD

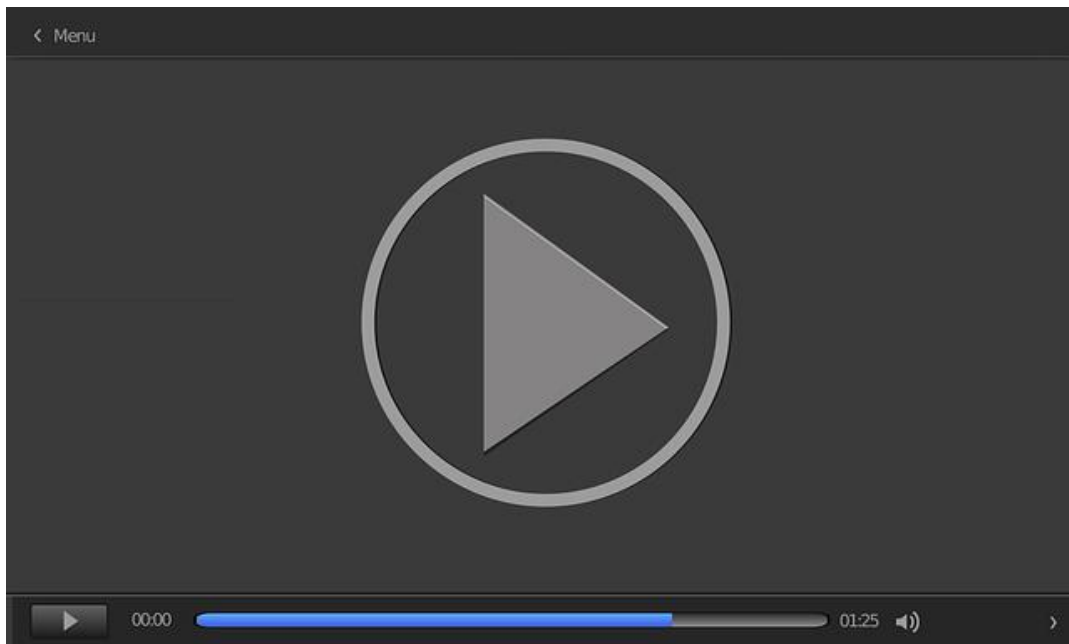
El proceso de creación de una BBDD no es complejo, pero con lleva bastantes pasos como has podido ver en el apartado anterior así que la mejor forma de ver como se crear una BBDD y repasar todo lo que hemos visto hasta ahora es a través del siguiente video, donde te voy a mostrar paso a paso como crear un BBDD y como asignar a cada campo el tipo de dato correspondiente y por qué.

Recuerda siempre que un modelo de datos es una estructura abierta y que depende mucho del modelo de negocio de la aplicación que estemos desarrollando, con lo que cualquier toma de decisiones tienes que validarla o bien con tus compañeros de trabajo o bien con el cliente para ver si cumple necesidades y sobre todo si el resto de tus compañeros de equipo entienden ese modelo y lo comparten.

En el siguiente video explico paso a paso todo esto y vemos un ejemplo de una aplicación real de una base de datos con una tabla y el proceso de razonamiento en la creación de cada campo.

En este vídeo, *Crear una BBDD con MySQL Workbench*, se muestra un ejemplo de una base de datos sencilla para la gestión de tareas, creación de tablas y registros a través de Workbench.

Tema 3. Cómo construir una base de datos



Accede al vídeo:

<https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=20df3a8d-d820-452b-a64a-aecf000d746e>

Tema 4. Modelo entidad relación

4.1. Introducción y objetivos

El objetivo de este tema es entender cómo funciona un **modelo o diagrama entidad relación en BBDD MySQL**, y como esta herramienta nos ayuda a diseñar y construir BBDD relacionales de una forma más fácil y cómoda.

También nos ayuda a entender y clasificar nuestras tablas de base de datos en función de cómo se relacionan entre ellas.

En este tema trataremos conceptos como que es una entidad, que tipos de relaciones se crean entre cada entidad y cómo manejar el concepto de cardinalidad. Estructuraremos cómo se genera una relación entre dos tablas usando los conceptos básicos de clave primaria y foránea

En definitiva, miraremos una base de datos MySQL como un conjunto de tablas que se relacionan y que me permiten trabajar de forma escalable y práctica.

Tema 4. Modelo entidad relación

4.2. ¿Qué es el concepto de entidad relacional y cardinalidad?

El objetivo de este tema es entender cómo funciona un modelo o diagrama entidad relación entre sí. Primero definamos qué es el concepto de entidad.

Una entidad es un conjunto de elementos relacionados entre sí. Pongamos unos ejemplos: alumnos, libros y productos son elementos que representan una entidad o tabla dentro de nuestra base de datos y cuyos registros tiene los mismos campos dentro de cada registro.

La **relación** es el elemento que nos permite unir dos entidades y relacionarlas entre sí para que la unión de estas me permita más cantidad de datos.

Cardinalidad es el modelo de relación entre dos entidades. En función de la relación que existe entre dos tablas tenemos tres tipos de cardinalidad.

- ▶ Relación: 1:1.
- ▶ Relación: 1:N (uno -> muchos).
- ▶ Relación: N:M (muchos -> muchos).

En esencia un modelo entidad relación o lo que se conoce como E-R, consiste en buscar las entidades o tablas que describan los elementos que intervienen en la resolución de un problema y como se relacionan con el resto de las tablas de mi sistema.

RELACIÓN 1:1

Una tabla o entidad se relaciona directamente con otra. Por ejemplo, si tuviésemos una tabla con distintos usuarios y otra con ordenadores deberíamos poder

Tema 4. Modelo entidad relación

determinar que cada usuario solo puede tener un ordenador (y cada ordenador un usuario). Esto genera un campo dentro de la tabla considerada como padre. Pueden también considerarse como un campo dentro de una misma tabla o entidad.

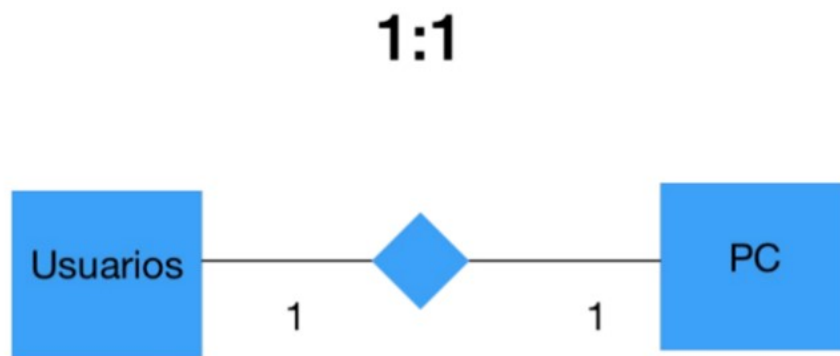


Figura 21. Relación 1-1. Fuente: elaboración propia.

RELACIÓN 1:N

Determina que un registro de una tabla puede estar relacionado con varios de otra tabla, pero en esta tabla existir solo una vez. Es decir, evita que para la creación de más elementos relacionados con la primera tabla los datos de esta queden duplicados.

En el ejemplo que vemos abajo una escuela puede tener N alumnos, pero un alumno solo puede estar en una escuela. Traducido a la creación de tablas la tabla que se representa con el 1 es el padre y la que se representa con N es el hijo, luego deberemos poner el FK (clave foránea) en el hijo con el ID que tiene el padre.

Tema 4. Modelo entidad relación

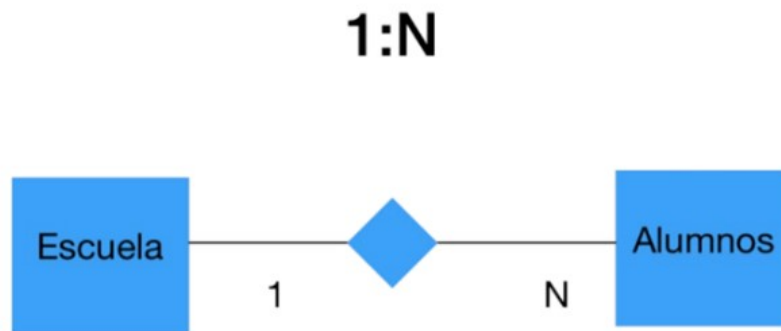


Figura 22. Relación 1-N. Fuente: elaboración propia.

RELACIÓN N:M (N:N)

En una relación muchos-muchos se determinan que una tabla puede relacionarse con otro con ninguno o varios registros y viceversa. Por ejemplo, un alumno puede tener varias materias dentro de un curso y una materia puede ser tenida por varios estudiantes, luego ambos pueden tener relación múltiple directa.

Cuando la relación es N:N deberemos crear una tabla entre las dos tablas, que se conoce como tabla intermedia ya que no se tiene una tabla padre y otra hija si no que las dos tiene la misma importancia, con lo que crearemos una tabla que hará de hijo de las dos tablas iniciales, esta tabla recibe el nombre de tabla de índices. (por que la mayoría de las veces solo almacena índices, aunque no tiene que ser siempre y puede almacenar algún otro tipo de dato o campo en el registro que lo complete).

Tema 4. Modelo entidad relación

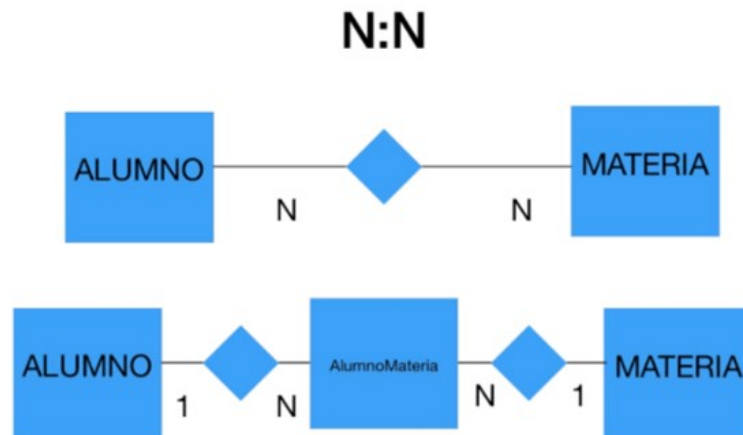


Figura 23. Relación N-N. Fuente: elaboración propia.

Dentro de un modelo entidad relación hay una parte muy importante que hay que conocer. Es el concepto de clave primaria y clave foránea, que nos sirve para identificar de forma inequívoca no solo los registros de una tabla (clave primaria o primary key), si no para relacionar con otro registro de otra tabla (clave foránea o foreign key)

- ▶ **CLAVE PRIMARIA (Primary key):** selecciona de forma inequívoca un registro en una tabla. Por lo general no acepta valores NULL y si es de tipo ID suele ser auto incremental para facilitar su creación EJ: DNI, matricula. ID
- ▶ **CLAVE FORANEA (FOREING key):** Es la llave con la que vamos a relacionar con al primary key de otra/as Tablas de nuestra base de datos.

Tema 4. Modelo entidad relación

4.3. Crear un modelo ER a través de MySQL Workbench y relacionarlo

MySQL Workbench es una herramienta muy potente con la que podemos crear bases de datos MySQL de una forma relativamente sencilla. Dentro de esta herramienta tenemos lo que se denomina creación de modelos, es una herramienta abstracta que me permite crear modelos de datos visuales y a partir de ahí generar nuestras bases de datos.

A través de un interfaz visual podrás generar diferentes modelos con sus tablas y crear la relación entre ellas, generando, de forma automática las fk's y los índices necesarios para justifica y bloquear esa relación.

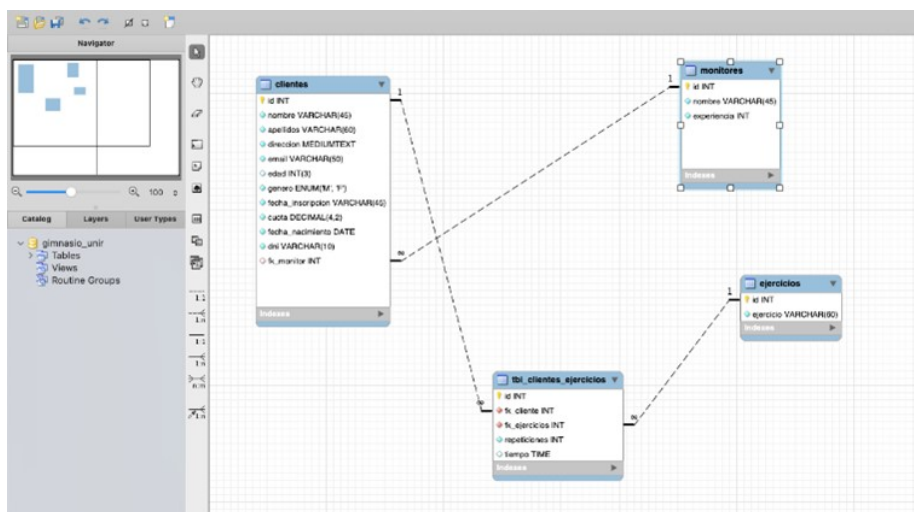


Figura 24. Modelo E-R MySQL Workbench. Fuente: elaboración propia.

Como ya hemos visto en el tema anterior crear una base de datos no requiere de usar este interfaz de modelos, pero tener esta herramienta me sirve para compartir fácilmente dicho modelo con otros compañeros y que podemos validarlo de una forma fácil **antes** de crear nuestra base de datos y llenarla con registros.

Tema 4. Modelo entidad relación

Además, también sirve como backup para nuestra estructura de datos, permitiendo no solo crear las veces que necesite de nuevo la base de datos con la estructura limpia, si no hacer modificaciones solo a nivel de tablas y modificar y volver a crear dicha tabla.

Como puedes ver en la imagen de arriba la herramienta me aporta un interfaz de pseudo-dibujo que me permite crear diferentes tablas para posteriormente definir los campos de esta.

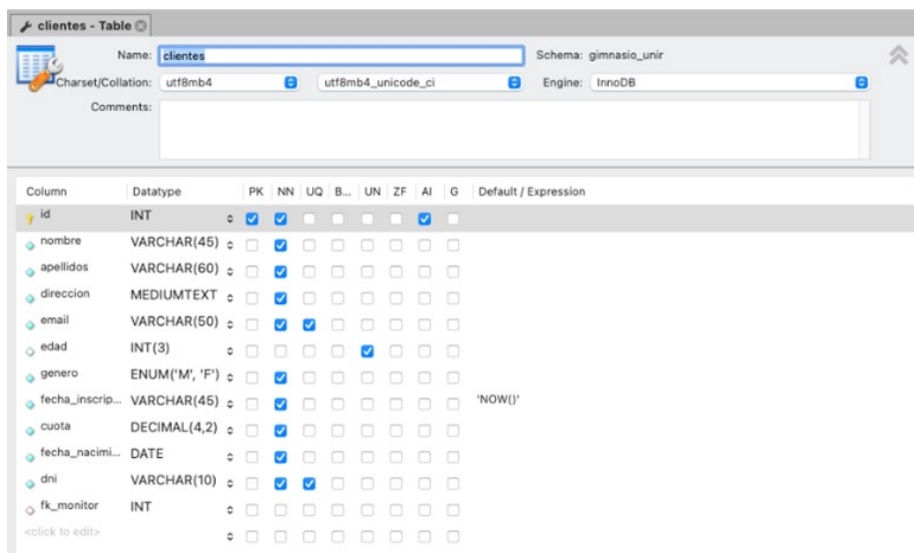


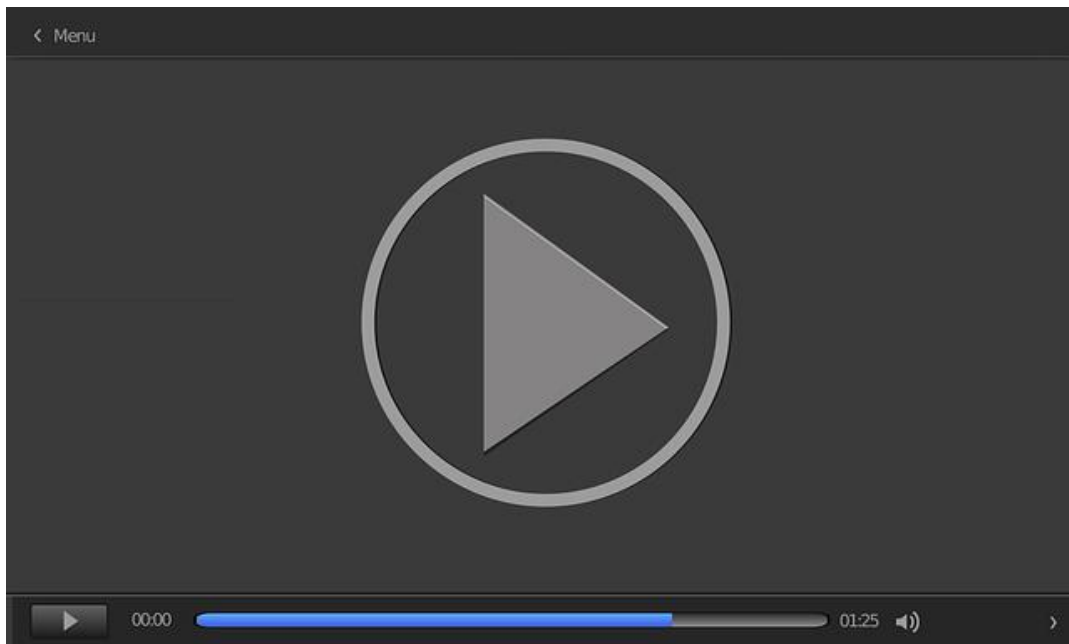
Figura 25. Creación de campos de tabla. Fuente: elaboración propia.

Posteriormente podremos crear con sus herramientas las relaciones necesarias entre tablas que nos ayudarán a crear de forma automática las fk de tabla necesarias dependiendo del tipo de relación.

En el siguiente vídeo te enseño como manejar esta potente herramienta que te va a permitir crear BBDD relacionales MySQL de una forma sencilla.

En el siguiente vídeo, Crear una BBDD con modelo relación con MySQL Workbench parte 1, se muestra un ejemplo de base de datos para la creación de un gimnasio, sacando las relaciones entre tablas.

Tema 4. Modelo entidad relación



Accede al vídeo:

<https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=5e0cb19c-ae2c-43e1-b23e-aecf000d7430>

Tema 4. Modelo entidad relación

4.4. Contrains o protección de relaciones

Lo primero de todo es puntualizar que es unas restricciones, como y donde se pueden dar.

Las restricciones sobre los elementos de una base de datos no tienen otro trabajo que garantizar que cierta estabilidad sobre los datos de una tabla y evitar que alguien, ya sea por error, o de forma malintencionada, pueda eliminar ciertos elementos.

Las restricciones se pueden dar a nivel de campo, ya sea por ejemplo evitando que una clave primaria pueda ser nula, o que un campo cualquier no pueda tener valor nulo.

Una de las restricciones, más comunes a nivel de campo es UNIQUE, esta restricción evita por ejemplo que un valor pueda repetirse en otro registro de la misma tabla, por ejemplo, un email o un DNI. Al ponerlo como UNIQUE evitamos la duplicación de datos.

Otro tipo de restricciones es la que se aplica a las relaciones y esta van ser en las que nos centremos a partir de ahora, ya que evitan que un elemento que esté relacionado con otro se pueda borrar o marca que si se borra como se debe comportar el elemento con el que está relacionado.

Según esto tenemos tres tipos de restricciones de relación, **cascade**, **set null**, **restrict**.

Para que dos tablas puedan relacionarse entre si el motor de almacenamiento debe ser INNODB.

A una clave foránea le podemos poner una restricción de cabla externa que nos bloquee acciones de DELETE Y UPDATE, protegiendo así la integridad referencial

Tema 4. Modelo entidad relación

de nuestra base de datos.

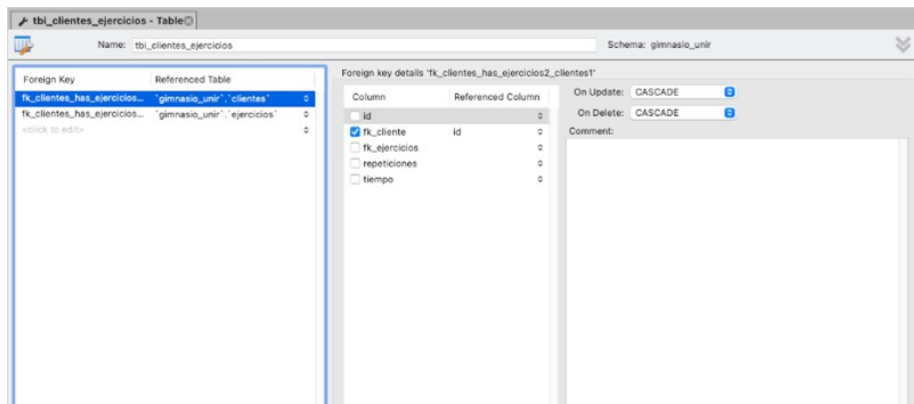


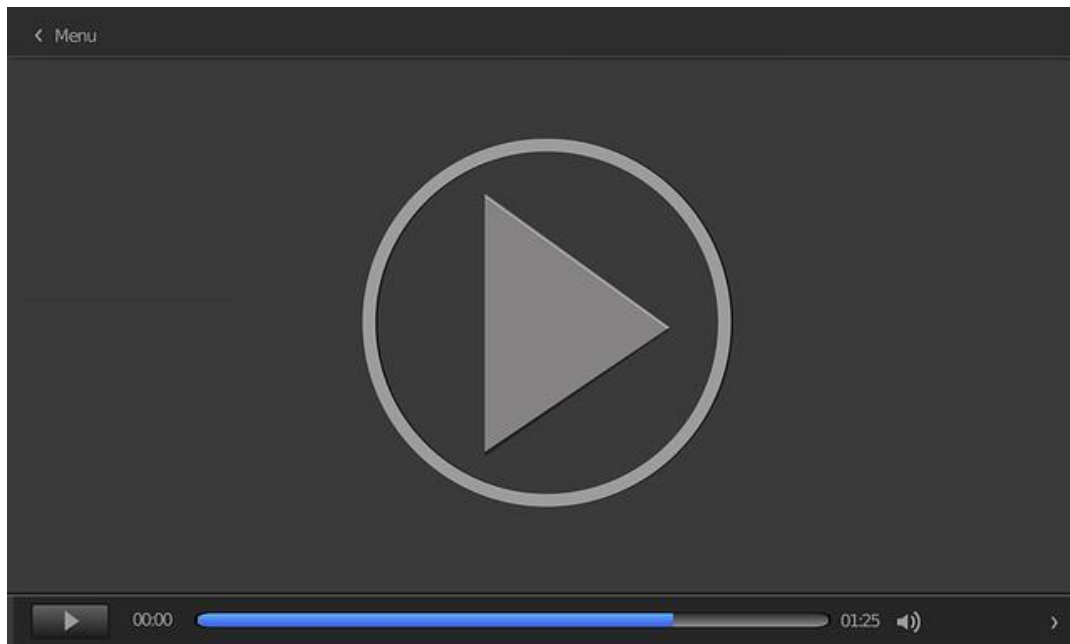
Figura 26. Contrains claves foraneas. Fuente: elaboración propia.

Veamos cada una de las posibilidades por separado.

- ▶ **RESTRICT:** Impide que se puede realizar cualquier tipo de modificación sobre un elemento que está relacionado con otro mientras existan registros dentro de las tablas y así proteger la integridad referencial. La base de datos no permitirá borrar registros.
- ▶ **CASCADE:** Lo que le ocurra al registro de la tabla principal también le ocurrirá al de la tabla dependiente, es decir que si borrar un registro el que esté relacionado también será borrado.
- ▶ **SET NULL:** Establece a NULL el valor de la clave foránea cuando se elimina el registro de la tabla principal o se modifica. Para poder realizar esto, la clave foránea tiene que permitir valores nulos.

El siguiente vídeo, Crear una BBDD con modelo relación con MySQL Workbench parte 2, es la continuación del vídeo anterior donde mostramos cómo llevar a cabo estas restricciones en nuestro modelo, y así proteger la identidad estructural de nuestra BBDD.

Tema 4. Modelo entidad relación



Accede al vídeo:

<https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=982928d5-ad80-4eac-822e-aecf000d7414>

Tema 5. Sentencias CRUD con MySQL

5.1. Introducción y objetivos

El objetivo de este tema es entender cómo funciona el lenguaje SQL con el que está formado MySQL y que me permite extraer información de una BBDD por medio de consultas también conocidas como QUERIES.

Las consultas se agrupan en varios tipos dependiendo de su funcionalidad dentro del lenguaje SQL.

Dentro del desarrollo web las sentencias más comunes son las que denominamos CRUD (Create, Read, Update, Delete): Creación, Lectura, Actualización y Borrado.

En este tema vamos a aprender estas sentencias básicas que luego asociaremos a nuestro servidor en backend para poder dotar de datos a nuestras aplicaciones.

Tema 5. Sentencias CRUD con MySQL

5.2. Creación, vaciado y borrado de tablas

Lo primero que vamos a aprender es como crear una BBDD a través de código SQL.

Para crear una BBDD solo tenemos que escribir lo siguiente.

```
CREATE DATABASE nombre_base_datos;
```

Posteriormente para usar y dejar activa esa BBDD tenemos que usar el siguiente comando:

```
USE nombre_base_datos;
```

Nótese que después de cada sentencia SQL ponemos punto y coma ya que si no la sentencia no se cierra y genera fallo.

Después de crear una BBDD tenemos que crear tablas y para ello vamos a usar la siguiente sentencia.

```
CREATE TABLE alumnos(  
  id INT(11) AUTO_INCREMENT PRIMARY KEY,  
  nombre VARCHAR(30) CHARACTER SET utf8mb4 COLLATE utf8mb4_general_ci NOT  
  NULL,  
  apellidos VARCHAR(30) NOT NULL,  
  nacimiento DATE,  
  promedio TINYINT,  
  sexo CHAR(1),  
  idEscuela TINYINT,  
  idClase TINYINT,  
  idCurso TINYINT  
);
```

Estas sentencias permiten crear, tanto una base de datos como una tabla dentro de la misma, pero si queremos borrar la tabla o vaciarla de contenido aplicaremos la siguiente instrucción.

```
DROP TABLE nombreTabla; //para BORRAR una tabla de una BBDD
```

Tema 5. Sentencias CRUD con MySQL

```
TRUNCATE TABLE nombreTabla; // para vaciar la tabla sin borrar
```

Ambos códigos son bastante útiles ya que en nuestro desarrollo nos veremos obligados a borrar pero sobre todo a vaciar a tablas de datos de prueba que no necesitemos.

Tema 5. Sentencias CRUD con MySQL

5.3. Insertar, borrar y actualizar un registro

Dentro de las sentencias CRUD las más sencillas son aquellas que consisten en insertar o borrar datos, seguidas de las de actualización ya que son siempre iguales dentro de la complejidad y de la cantidad de campos a registrar.

Consulta de inserción de un registro en una base de datos:

```
INSERT INTO tabla (columnas) VALUES (valores);
```

Si aplicamos la sentencia a consultas reales tenemos lo siguiente.

```
INSERT INTO alumnos (nombre, apellido, nacimiento, promedio, sexo,
idEscuela, idClase, idCurso) VALUES ("Paco", "Arce", "1989-02-10", 8, "M",
1, 1,1);
```

Esta consulta nos permite insertar dentro de cualquier tabla todos los campos propuestos dentro de la consulta.

Esta consulta se puede acortar. Se pueden omitir los nombres de los campos, pero MySQL los insertará en orden en que aparecen los campos en la tabla. Hay que tener cuidado al respecto:

```
INSERT INTO tabla VALUES (valores);
INSERT INTO alumnos VALUES (0, "Paco", "Arce", "1966-02-10", 8, "M",1,1,1);
```

Si tomamos la decisión de omitir los nombres de los campos tenemos que tener en cuenta que dichos campos deben ir en el orden establecido en la creación de tabla, ya que MySQL los insertará en dicho orden.

Sentencia para actualizar valores dentro de una tabla de la base de datos

Tema 5. Sentencias CRUD con MySQL

En muchas ocasiones es necesario modificar los datos que ya se encuentran en nuestras tablas. Para ello contamos con la siguiente sentencia:

```
UPDATE tabla SET columna=valor [,columna2=valor2...] [WHERE condición]
[LIMIT n]
```

Ej:

```
UPDATE contactos SET email="paco@mail.com" WHERE nombre="Paco";
```

Donde SET contiene una lista separada por comas de las columnas que deben actualizarse y el nuevo valor de cada columna, con el formato columna=valor.

La cláusula WHERE en la sentencia UPDATE es opcional, y si se omite, entonces se actualizan TODAS las filas de la tabla destino.

WHERE: Especifica la condición de búsqueda que define las filas de las tablas de origen que están calificadas para proporcionar valores para las expresiones de la cláusula SET.

LIMIT n establece en n el número máximo de filas que se pueden actualizar.

Ojo hay que tener mucho cuidado, debido a que puede cambiar en forma definitiva la información de sus tablas, siempre utilice el **UPDATE** con la sentencia **WHERE** (a menos que desee cambiar todas las columnas por un mismo valor).

Consulta para borrar registros dentro de una tabla

Para **BORRAR** un registro registro, o grupos de registros, se utilizará la sentencia **DELETE**:

```
DELETE FROM tabla [WHERE condicion] [LIMIT n]
```

En esta instrucción también es necesario utilizar la sentencia **WHERE**, menos que

Tema 5. Sentencias CRUD con MySQL

desea **borrar todos los datos de la tabla**, por ejemplo:

```
DELETE FROM contactos WHERE nombre = "Paco";  
DELETE FROM contactos WHERE id = 12;
```

En este último caso la borra por un identificador único me permite controlar de forma correcta que el elemento borrado es ese y no otro, en el primer ejemplo se pueden borrar varios elementos cuyo nombre coincida con Paco.

Tema 5. Sentencias CRUD con MySQL

5.4. Select, la potencia de las consultas dentro un BBDD

Para hacer consultas de petición de datos tenemos la sentencia SELECT, esta es la que más juego da por que permite extraer información de una o más tablas, organizarlas, ordenarlas, y muchas cosas más a través de condiciones y usos de la propia consulta.

En el video que esta al final de este tema analizaremos estas mismas sentencias pero con una explicación directa para que sea más fácil comprender lo que estamos implementando en este documento.

Vamos a usar como referencia la base de datos del gimnasio que creamos en el ejercicio anterior para tener un conjunto de datos suficientemente real para hacer consultas de selección y poder realizar diferentes y variados tipos de consultas.

Siguiendo este esquema:

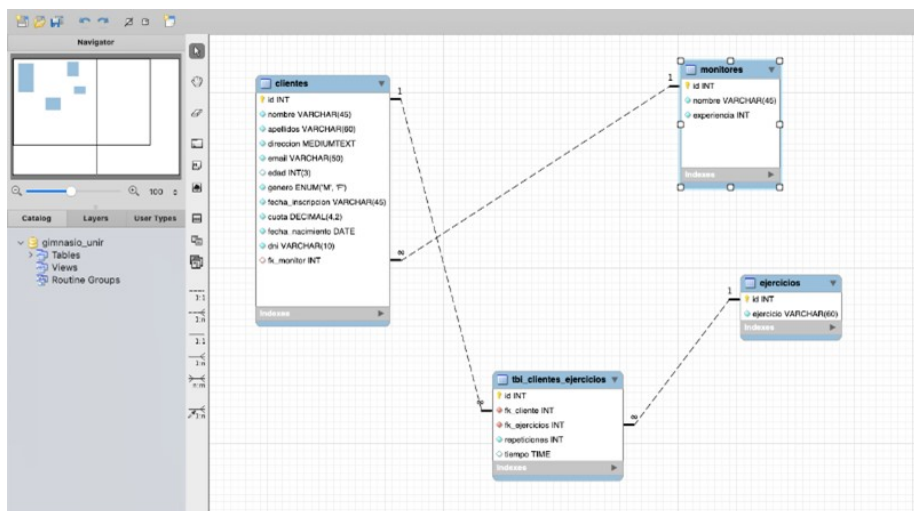


Figura 27. Modelo entidad Relación del ejemplo. Fuente: elaboración propia.

Vamos a ir poniendo ejemplos de sentencias SELECT en base a este modelo de

Tema 5. Sentencias CRUD con MySQL

datos para que vaya teniendo sentido y podemos ir viendo la relación entre lo que me piden y como se ejecuta en MySQL, una vez tengas montada la base de datos te invito a que sigas a paso a paso cada consulta para que veas el resultado de esta y entiendas su resultado.

Seleccionar todos los datos de una tabla cualquiera

```
SELECT * FROM gimnasio.clientes;
```

La primera parte de la consulta el * significa que queremos todos los campos y después del FROM seleccionamos la tabla de la que lo queremos, poner el nombre de la BBDD gimnasio delante no es obligatorio, pero si aconsejable ya que tablas clientes puedes tener varias de esta forma está especificando que es la tabla clientes de la BBDD gimnasio.

Seleccionar solo el nombre y el apellido de la tabla clientes

```
SELECT clientes.nombre, clientes.apellidos FROM gimnasio.clientes;
```

A continuación del SELECT estas especificando que quieres el campo nombre de la tabla clientes y el campo apellido de la tabla clientes. El poner clientes delante te ayudará para cuando haya que hacer consultas a más de una tabla y quieras especificar de que nombre se trata.

Alias de campo y de tabla

Los alias son ayudas necesarias a nuestra escritura de código MySQL ya que nos permite acortar o renombrar tanto tablas como campos para que el resultado final de la consulta salga de determinada forma.

Tema 5. Sentencias CRUD con MySQL

```
SELECT clientes.nombre, clientes.fecha_inscripcion as 'inscripcion' FROM
gimnasio.clientes;
```

La sentencia anterior en la salida de la consultad nos mostrará en lugar de la fecha_inscripcion el campo inscripcion. Notase que ambos campos no tienen acentos, ya que debemos evitar estos caracteres dentro del nombre tanto de tablas como de campos.

```
SELECT c.id, c.nombre, c.fecha_inscripcion FROM gimnasio.clientes as c;
```

La sentencia anterior la tabla clientes la renombramos como **c** esto hace que la consulta se acorte ya que una vez que hemos puesto ese alias siempre que nos refiramos a dicha tabla ya la marcaremos como **c** en toda la consulta.

```
SELECT c.apellidos, c.edad, c.cuota as 'mensualidad' FROM gimnasio.clientes
as c;
```

Funciones dentro de una consulta

MySQL tiene muchas funciones dentro del lenguaje que nos ayudan a realizar acciones que de otra forma serían mucho más complicadas, aquí vamos a ver algunas de ellas y como se utilizan, el final de pondremos una lista de las más comunes dentro del MySQL y para qué sirven.

Las funciones se aplican sobre los campos que queremos obtener ya que son acciones que queremos aplicar, por ejemplo, sobre el total de un campo o sobre elementos que se repiten.

COUNT(): nos permite contar cuantos elementos únicos hay en una tabla

Pongamos un ejemplo para explicar cómo funcionan este tipo de acciones y luego un listado con diferentes funciones y lo que realiza cada una.

Tema 5. Sentencias CRUD con MySQL

```
SELECT COUNT(clientes.sexo) FROM gimnasio.clientes WHERE clientes.sexo = "F";
```

En este caso COUNT cuenta los clientes por género únicos, por ejemplo, masculino "m" y femenino "f", en el ejemplo queremos saber cuántos clientes de nuestro gimnasio son mujeres.

Si quisiéramos obtener, por ejemplo, la edad media de los clientes de nuestro gimnasio usaríamos la función AVG.

```
SELECT AVG(clientes.edad) as 'edad_media' FROM gimnasio.clientes;
```

Como último ejemplo de este tipo de funciones tenemos el valor MAX() "máximo" y MIN() "mínimo".

Como ejemplo podemos usar estas funciones para saber la edad máxima y mínima de nuestros clientes, en este caso usaremos el alias de campo para crear este nuevo elemento de mi consulta.

```
SELECT MAX(clientes.edad) as 'edad_maxima', MIN(clientes.edad) as 'edad_minima' FROM gimnasio.clientes;
```

Otras funciones interesantes de MySQL (no todas las que tiene por supuesto) son:

- ▶ CEIL() redondeo a la alta.
- ▶ FLOOR() redondeo a la baja.
- ▶ ROUND() redondeo matemático.
- ▶ TRUNCATE(numero, decimales) - TRUNCATE(3.14151617, 2) -> 3.14.
- ▶ POWER(numero, exponente) - POWER(2,3) -> 8.
- ▶ SQRT() -> raíz cuadrada.

Tema 5. Sentencias CRUD con MySQL

WHERE para condicionar nuestras consultas

La sentencia **WHERE**, nos sirve para hacer sentencias condicionadas, es decir darle un campo que es condicionado dentro de nuestra petición, por ejemplo, que los clientes tengan una edad menor de 30 años.

```
SELECT * FROM gimnasio.clientes WHERE clientes.edad = 30;
```

Operadores condicionales de la cláusula WHERE MySQL

Veamos los operadores condicionales:

- ▶ **=** El operador **=** determina si el valor de los campos comparados es **igual**.
- ▶ **!=** El operador **!=** determina si el valor de los campos comparados **son distintos**.
- El operador **>** determina si el primer valor es **mayor** que el segundo valor comparado.
- ▶ **<** El operador **<** determina si el primer valor es **menor** que el segundo valor comparado.
- ▶ **>=** El operador **>=** determina si el primer valor es **mayor o igual** que el segundo valor comparado.
- ▶ **<=** El operador **<=** determina si el primer valor es **menor o igual** que el segundo valor comparado.

Si quiero concatenar varias condiciones tendré que usar operadores lógicos como **AND Y OR** que me ayudarán de concatenar dos o más condiciones como si de una sentencia condicional se tratara.

```
SELECT * FROM gimnasio.clientes WHERE clientes.edad >= 40 AND clientes.edad
```


Tema 5. Sentencias CRUD con MySQL

`<= 60;`

En este ejemplo podéis ver tanto estructuras condicionales como lógicas encadenado varias condiciones. Estas no tienen que ser necesariamente del mismo campo.

Subconsultas

Este tipo de consultas me permiten preguntar a través de pequeñas peticiones un dato que, aunque tengo solo podría determinar haciendo primero una consulta y luego otra. Las subconsultas me permiten hacer dentro de una consultad principal otra para conseguir un campo necesario para la consulta principal

Por ejemplo, en este caso queremos saber todos los datos de los clientes de un profesor que se llama Nora Contreras, en la tabla clientes está relacionado a través de la clave foránea `fk_monitor` pero no sabemos su id solo su nombre así que haremos una subconsulta.

```
SELECT * FROM gimnasio.clientes WHERE fk_monitor = ( SELECT monitores.id
FROM gimnasio.monitores WHERE monitores.nombre = "Nora Contreras");
```

La primera es la consulta principal y la que está entre paréntesis es la subconsulta que me devuelve el id de la profesora "Nora Contreras" y así usarlo para compararlo con la `fk_profesor` que relaciona cliente con profesor.

Otro ejemplo podría ser sacar todos los clientes que sean mayores que la media de edad del gimnasio.

```
SELECT * FROM gimnasio.clientes WHERE clientes.edad >= ( SELECT
AVG(clientes.edad) FROM gimnasio.clientes);
```

En esta última consulta concentramos muchas cosas de las aprendidas hasta ahora como son la sentencia **WHERE** las **SUBCONSULTAS** y las **FUNCIONES**, según

Tema 5. Sentencias CRUD con MySQL

vayamos avanzando las consultas irá siendo cada vez de mayor longitud.

Ordenar Resultados - ORDER BY ASC/DESC

La cláusula ORDER BY se usa para ordenar los conjuntos de resultados de la consulta en orden ascendente o descendente. Se usa junto con la consulta SELECT. Tiene la siguiente sintaxis básica.

```
SELECT * FROM gimnasio.clientes ORDER BY clientes.edad DESC;
```

Para ordenar numéricamente es sencillo pensar que DESC quiere decir de 10 a 1 y ASC quiere decir de 1 a 10 (los números son solo ilustrativos, se puede ordenar desde cualquier cantidad).

Es en el ámbito alfabético donde es más difícil ver que es ASC y que es DESC. Un truco o regla nemotécnica sería lo siguiente cuando queramos ordenar de A-Z empezamos por A luego ASC y al contrario DESC de la Z-A

Esta consulta se puede encadenar con otras ordenaciones. Por ejemplo, quiero una lista de clientes ordenados por edad de mayor a menor y en caso de tener la misma edad me las ordenas de por nombre de la A-Z

```
SELECT * FROM gimnasio.clientes ORDER BY clientes.edad DESC,  
clientes.nombre ASC;
```

Tema 5. Sentencias CRUD con MySQL

Consultas a más de una tabla

Se pueden hacer consultas a más de una tabla de forma fácil en MySQL solo hay que especificar las dos tablas a las cuales se consultan en la parte del FROM.

Por ejemplo, si quisiera los datos de nombre, cuota del cliente y junto con el nombre del monitor que da clase. Esta consulta los datos están en dos tablas y **¡¡ojo!!** los campos nombre existen en las dos tablas monitor y cliente, con lo que tendremos que diferenciarlos con alias de campo.

```
SELECT c.nombre, c.cuota, m.nombre as 'profesor' FROM gimnasio.clientes as c, gimnasio.monitores as m WHERE c.fk_monitor = m.id;
```

Usamos el alias de campo para diferenciar entre el nombre del profesor y el del cliente, los alias de tabla los ponemos para hacer más corta la consulta y por último **siempre que tenemos dos tablas como sentencia condicional hay que poner la relación entre ambas tablas.**

```
SELECT c.nombre, c.email, c.cuota, m.nombre as 'profesor', m.experiencia FROM gimnasio.clientes as c, gimnasio.monitores as m WHERE c.fk_profesor = m.id AND m.experiencia > 10;
```

Ejemplo de una lista de clientes con nombre, email, cuota, y nombre del profesor, experiencia, pero solo aquellos datos cuyo profesor tenga mas de 10 años de experiencia

Si quisiera todos los datos de una tabla junto con todos o algunos datos de otra tabla lo que tendría que hacer es colocar el nombreTabla.* como campo que pido en la consulta. Veamos un ejemplo:

```
SELECT clientes.*, monitores.* FROM gimnasio.clientes, gimnasio.monitores WHERE clientes.fk_monitor = monitores.id;
```

Hasta aquí diferentes tipos de consultas, todo este material escrito también lo tienes

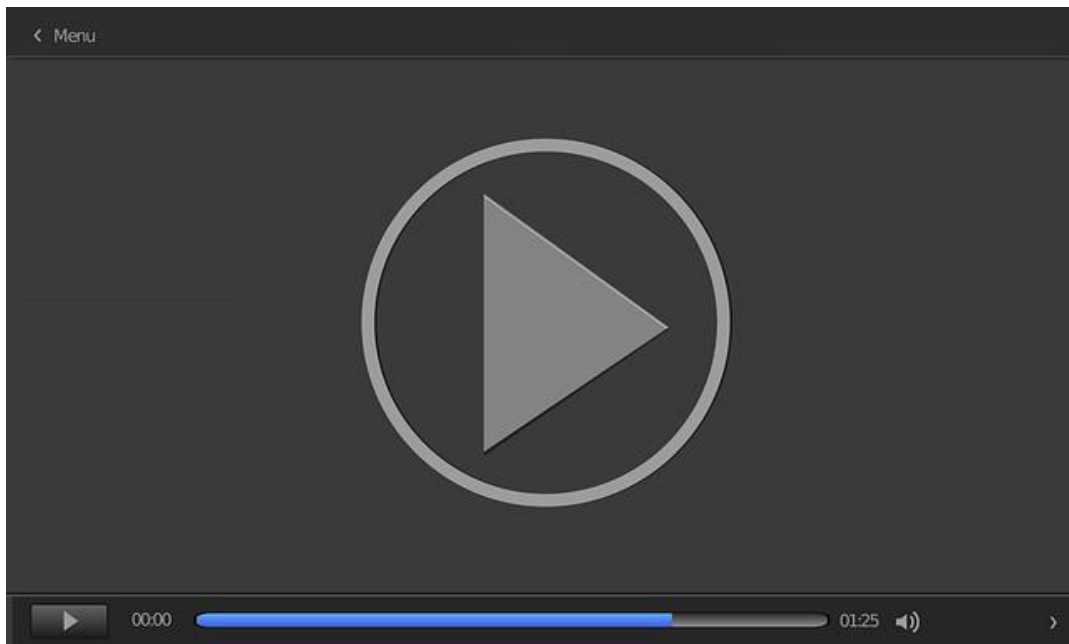
Tema 5. Sentencias CRUD con MySQL

disponible en el siguiente video. Te servirá para aprender a aplicar todos estos conceptos y mirar cómo se llevan a cabo consultas de este tipo.

Durante las clases presenciales profundizaremos más sobre este tipo de consultas y veremos algunas nuevas que nos permitan mejorar mucho los resultados obtenidos de una BBDD.

En el siguiente vídeo, *Consultas a la BBDD del gimnasio creada en el video anterior*, se muestran diferentes tipos de consultas a la BBDD de gimnasio que creamos en el vídeo anterior.

Tema 5. Sentencias CRUD con MySQL



Accede al vídeo:

<https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=0de48088-b3fe-436c-837f-aecf000e93e8>

Tema 7. Introducción y Diseño de estructuras de BBDD

7.1. Introducción y objetivos

Cuando nos adentramos en el mundo de las bases de datos NoSQL es importante establecer una serie de conceptos que nos separen del conocimiento que podemos tener de bases de datos tradicionales como MySQL.

En nuestro camino para convertirnos en desarrolladores web full stack vamos a tener que enfrentarnos a muchos tipos de bases de datos diferentes y en este tema vamos a intentar comparar la manera que tenemos de diseñar tanto unas como otras. Para agilizar el proceso intentaremos comparar la manera que tenemos de trabajar en bases de datos más tradicionales con los conceptos que utilizaremos cuando nos enfrentemos a una base de datos no relacional.

No es cierta la afirmación que dice que no debemos diseñar la estructura de nuestras bases de datos NoSQL. Es verdad que nos ofrecen mayor flexibilidad a la hora de especificar los datos que vamos a usar, incluso cierto dinamismo en la definición de tipos, pero no podemos dejar de lado los pasos previos que nos permiten organizar cómo vamos a interactuar con dichas bases de datos.

Tema 7. Introducción y Diseño de estructuras de BBDD

7.2. Diseño de bases de datos NoSQL

Para el diseño de bases de datos relacionales como MySQL existen multitud de herramientas que nos proporcionan una base para poder diseñar con garantías totales bases de datos que posteriormente utilizaremos en nuestros desarrollos. El claro ejemplo de este tipo de herramientas es Mysqlworkbench y la posibilidad que nos ofrece para poder crear modelos de datos.

Este tipo de software no existe tan avanzado para el diseño de bases de datos NoSQL, pero no por ello, como ya hemos comentado, debemos dejar de lado el diseño previo de nuestros datos.

A diferencia de las bases de datos tradicionales, las bases de datos NoSQL fomentan patrones de diseño basados en «Application first» o API-first. Antes de plantearnos cualquier tipo de acción sobre la base de datos debemos tener en cuenta qué vamos a necesitar para nuestra aplicación o cuáles son los requerimientos de nuestros proyectos.

Esta forma de trabajar se encuentra muy próxima al trabajo que hace el desarrollador y es por eso por lo que este tipo de bases de datos tomaron tanta fama en sus primeras versiones.

Los modelos que vamos a generar dentro de nuestras bases de datos no relacionales suelen ser bastante flexibles y se delega su creación y mantenimiento a los propios desarrolladores a cargo de la aplicación. La estructura más rígida y compleja de una base de datos relacional obliga en muchos casos a la existencia de un equipo dedicado únicamente a la creación y mantenimiento de la base de datos.

Esta forma de trabajar acelera bastante los ciclos de desarrollo ya que no existen equipos intermedios entre el desarrollo y los datos que vamos a manejar. Este dinamismo permite al desarrollador agregar nuevas funcionalidades sin depender de

Tema 7. Introducción y Diseño de estructuras de BBDD

migraciones dentro del sistema.

Uno de los primeros conceptos que debemos tener en cuenta cuando diseñemos nuestra base de datos NoSQL es el nivel de consistencia que necesitamos para los datos. Por ejemplo, si estamos tratando con un sistema financiero el nivel de consistencia tiene que ser máximo, porque la pérdida de unos pocos datos puede provocar descuadres económicos importantes. Por contra, si estamos almacenando muestras de datos para realizar algún tipo de cálculo medio, la consistencia de nuestro diseño puede ser más baja ya que la pérdida de algún dato no compromete el buen funcionamiento de la aplicación.

Como ya hemos comentado, es importante que los datos almacenados en una base de datos NoSQL se encuentren organizados del mismo modo que van a ser consumidos por la aplicación. En este caso, si necesitamos alguna relación dentro de la información que estamos almacenando, en este tipo de bases de datos, está totalmente permitida la duplicidad de información.

Lo que buscamos es que el acceso a dichos datos sea lo más rápido posible y que ya se encuentren listos para ser consumidos por las diferentes aplicaciones cliente que accedan a ellos. Si tenemos que insertar un elemento varias veces podemos hacerlo ya que no disponemos de los JOIN entre tablas.

En bases de datos NoSQL, en vez de hacer separación de los diferentes elementos en tablas diferentes y posteriormente unirlos en la recuperación lo que se intenta es disponer de diseños de datos jerárquicos donde con una única consulta podamos recuperar toda la información necesaria.

Estas estructuras jerárquicas donde encontramos nuestra información las denominamos **Documentos** y las agrupaciones de muchos documentos son denominadas **Colecciones**.

Tema 7. Introducción y Diseño de estructuras de BBDD

7.3. Concepto de documento y colección

Es importante, para posteriormente poder trasladarlo a nuestros desarrollos, saber cómo están organizadas las bases de datos en MongoDB, para ello vamos a hacer un análisis de los conceptos de base de datos, colección y documento y así poder conocer cómo se organiza desde la base.

Cuando hagamos la instalación de MongoDB en nuestro ordenador, vamos a instalar un Servidor de MongoDB, el cual puede contener dentro la cantidad de bases de datos que necesitemos. Nuestro servidor, en un principio, se encontrará instalado dentro de nuestra máquina local, pero también nos podemos encontrar servidores online y, por lo tanto, la forma de conexión será totalmente diferente.

Como ya hemos comentado, dependiendo del número de proyectos que vayamos a llevar a cabo, un servidor de MongoDB puede contener todas las bases de datos que necesitemos. Bases de datos que serán independientes entre sí, es decir, no comparten la información que almacenamos en unas u otras.

Dentro de cada una de estas bases de datos podremos incluir una o más colecciones. Si hacemos un paralelismo con las bases de datos MySQL, la colección equivaldría a lo que conocemos como tabla, el almacén de nuestros datos.

Y, por último, en nuestra escala jerárquica encontramos que las colecciones pueden incluir multitud de documentos. Estos documentos sí que conforman los diferentes datos que vamos a almacenar dentro de la base de datos en cuestión.

Uno de los puntos importantes y también diferenciadores con otro tipo de bases de datos es que la estructura de base de datos, colección y documento se genera de manera implícita cuando nosotros empezamos a usar nuestras bases de datos. En otros casos como MySQL, el primero paso que debemos dar cuando queremos empezar a trabajar con la base de datos es la generación de la estructura, eso en

Tema 7. Introducción y Diseño de estructuras de BBDD

MongoDB no es necesario. Veremos que, si hacemos una inserción de un documento con datos concretos, la colección donde lo colocamos y la base de datos que lo contiene se generan de manera automática.

La simpleza de este concepto nos permite que, desde el propio desarrollo que estemos haciendo, se pueda lanzar cualquier tipo de interacción sobre la base de datos sin depender de scripts previos o cargas pesadas de datos.

Veamos un ejemplo que nos permita diferenciar la estructura que crearíamos dentro de una base de datos relacional con respecto a una base de datos MongoDB.

Vamos a dibujar el diseño de la base de datos relacional para poder almacenar datos de Clientes y Proyectos.

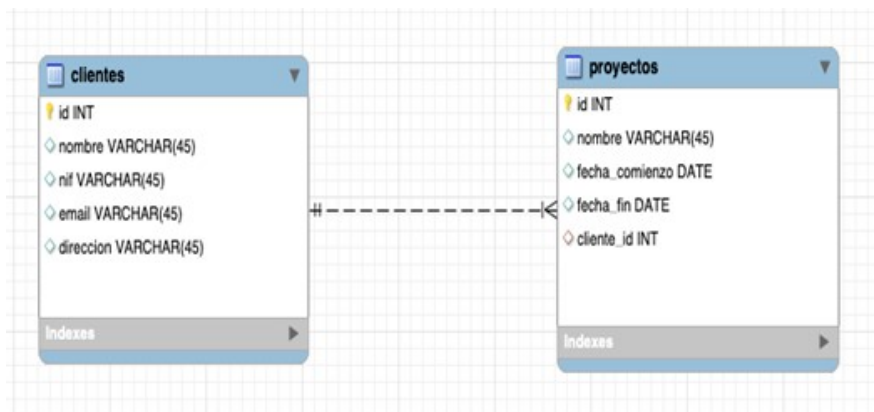


Figura 28. Diseño de modelos: clientes y proyectos. Fuente: elaboración propia.

En el esquema podemos apreciar cómo las dos tablas serían totalmente independientes. Cada una define las propiedades que vamos a insertar para los diferentes registros que almacenaremos. Aparte, hemos creado una relación entre ambas tablas determinando así que cada proyecto pertenece a un único cliente, mientras que un cliente puede disponer de múltiples proyectos.

Si insertamos datos dentro de dichas tablas, podríamos observar el siguiente comportamiento:

Tema 7. Introducción y Diseño de estructuras de BBDD

id	nombre	nif	email	direccion
1	Telefonica	81728891F	hola@telefonica.com	C/ Gran Vía 23, Madrid
2	Amazon	23433219K	contacto@amazon.es	C/ Blasco de Garay 25, Madrid
3	Ebay	87237540U	info@ebay.es	C/ Ballesta 54, Madrid
4	Corte Inglés	129883992I	tiendas@coraingles.com	C/ Preciados 58, Madrid

Figura 29. Datos incorporados dentro de la tabla clientes. Fuente: elaboración propia.

id	nombre	fecha_comienzo	fecha_fin	cliente_id
1	Desarrollo Front	25-1-22	30-3-22	2
2	Creación API	3-4-22	27-9-22	3
3	Repositorio código	30-5-22	2-6-22	2
4	Refactorización	23-6-22	17-10-22	1

Figura 30. Datos incorporados dentro de la tabla proyectos. Fuente: elaboración propia.

Apreciamos cómo los datos se integran dentro de la base de datos generando un registro para cada uno de ellos en la tabla correspondiente. Todos los registros disponen de los mismos campos y mantienen una estructura de tipos común.

Además, en la tabla de proyectos podemos enlazar mediante el campo `cliente_id` cuál es el cliente al que pertenece dicho proyecto.

Si esta misma estructura de datos la trasladásemos a una base de datos de MongoDB podríamos definirlo de la siguiente manera:

Tema 7. Introducción y Diseño de estructuras de BBDD

```
{
  nombre: 'Telefonica',
  nif: '81728891F',
  email: 'hola@telefonica.com',
  direccion: 'C/ Gran Vía 23, Madrid',
  proyectos: [
    {
      nombre: 'Refactorización',
      fecha_comienzo: '23-6-22',
      fecha_fin: '17-10-22'
    }
  ]
}
```

Figura 31. Representación de datos dentro de una colección de MongoDB. Fuente: elaboración propia.

Cada uno de los registros contenidos en la tabla relacional será representado a partir de un objeto dentro de la base de datos no relacional. Como podemos observar, la estructura es igual a lo que nos podríamos encontrar si trabajamos con objetos en Javascript.

En este caso el enlace entre las dos tablas se realizaría mediante la inclusión de los datos de la tabla hija dentro de la tabla padre. En MongoDB no es tan importante el tema de la duplicidad de los datos y es por ello por lo que nos podemos encontrar multitud de información en diferentes colecciones.

Esta relación, como ya veremos más adelante, también se podría hacer a partir del identificador único del documento dentro de la colección.

En definitiva, toda la estructura que encontremos dentro de nuestras bases de datos MongoDB será totalmente compatible con el código que vamos a manejar dentro de nuestras aplicaciones desarrolladas con JavaScript.

Tema 7. Introducción y Diseño de estructuras de BBDD

JSON vs BSON

Además de establecer estos principios en cuanto a la estructura se refiere, también debemos destacar el formato en el que se almacenan los datos. En este caso, no podríamos hablar de ficheros JSON como los que estamos acostumbrados a usar, si no que tendríamos que determinar las características del formato que realmente estamos usando en MongoDB, **BSON (Binary Json)**.

Para los desarrolladores, el uso de este tipo de ficheros es totalmente transparente ya que, cuando realicemos diferentes operaciones contra la base de datos, recuperaremos por defecto dichos datos en formato JSON para poder tratarlos desde nuestras aplicaciones.

Los **drivers de MongoDB** se encargan de transformar la información en formato JSON a datos binarios para poder almacenarlos. Esta transformación se hace buscando la eficiencia y la velocidad a la hora de almacenar esta información.

Además, estos datos transformados soportan una serie de tipos que no podemos utilizar en ficheros JSON normales. Por ejemplo, ya veremos que los identificadores de cada uno de los documentos contenidos en una colección son del tipo **ObjectId**, el cual no está soportado.

Cuando empecemos a insertar datos veremos cómo MongoDB se encarga internamente de ordenar toda esta información de la manera más eficiente y de transformar los tipos necesarios o incluso colocar los datos numéricos dependiendo de la precisión con la que los tratemos.

Este tipo de archivos BSON no tienen que seguir ningún tipo de esquema y el único requisito indispensable es que dispongan de una propiedad **_id** que sea única dentro de la base de datos donde estemos trabajando. Esta propiedad puede ser autogenerada o le podemos concretar nosotros el valor siempre y cuando mantengamos su unicidad.

Tema 7. Introducción y Diseño de estructuras de BBDD

Tema 7. Introducción y Diseño de estructuras de BBDD

7.4. Buenas prácticas

El uso de una base de datos MongoDB implica que podemos ser muy flexibles en cuanto su organización y estructura se refiere. Es por ello por lo que debemos seguir una serie de buenas prácticas para que nuestros desarrollos lleguen siempre a buen puerto.

Uno de los puntos más importantes a la hora de gestionar este tipo de bases de datos es **la creación de índices**. Resultan muy importantes para mejorar el rendimiento. Si vamos a realizar búsquedas de manera muy habitual sobre una colección de datos, lo óptimo es acompañar esa búsqueda a través de diferentes índices.

Por el contrario, no debemos excedernos en la creación de estos índices si no vamos a sacarles partido, ya que ocupan espacio y también pueden ser una fuente de problemas para nuestras bases de datos si los creamos sin criterios aparentes.

En todos los documentos ya contamos con el índice por defecto que agrega MongoDB de manera automática, pero nosotros como desarrolladores estamos obligados a mejorar el rendimiento de nuestras colecciones a través de la creación de otros totalmente diferentes.

Cuando nos enfrentamos a una base de datos MongoDB es **muy recomendable tener ciertas réplicas de los datos**.

Esto es algo que a los desarrolladores nos va a afectar poco y es una labor más específica del responsable de sistemas del proyecto. Es muy importante que nuestras bases de datos vayan replicando los datos insertados o modificados para poder afrontar cualquier caída de un nodo en concreto. Si nuestra base de datos se automatiza para replicarse en múltiples nodos, podemos recurrir a cualquiera de ellos en caso de necesidad.

Tema 7. Introducción y Diseño de estructuras de BBDD

Las bases de datos de MongoDB **funcionan mejor si toda la colección de datos y los índices asociados pueden trabajar dentro de la memoria del servidor donde se encuentran.**

Debemos ser capaces de ampliar la memoria destinada a nuestra base de datos para hacer frente a todos los datos contenidos dentro de la misma y así procurar un mejor rendimiento en memoria. Esto no quiere decir que sea totalmente obligatorio que la ejecución de la base de datos se encuentre en la memoria, pero sí quiere decir que baja mucho el rendimiento si tiene que interactuar con el disco duro.

El mantenimiento de nuestra base de datos es importante y si observamos que existe una sobre carga mayor de lo habitual debemos actuar en consecuencia escalando el sistema.

8.1. Introducción y objetivos

Una vez analizados los conceptos básicos y características sobre este tipo de bases de datos es el momento de ponerse manos a la obra y empezar a interactuar con ellas de una manera mucho más directa para que podamos observar de primera mano todas estas herramientas que tenemos disponibles.

Antes de poder utilizar una base de datos cualquiera dentro de nuestras aplicaciones debemos ser capaces de analizar y tratar con los aspectos más básicos de las mismas. Quizá como desarrolladores web, nuestro trabajo no siempre estará orientado a la gestión o mantenimiento de bases de datos, pero sí ayuda mucho que conozcamos cómo funciona para sacarle el máximo partido.

Este tema de transición lo utilizaremos para dejar instaladas todas las herramientas que utilizaremos a partir de ahora para gestionar nuestras bases de datos, ya sea a través de la consola o con ciertos gestores gráficos.

8.2. Instalación de MongoDB para sistemas Windows

La instalación para sistemas Windows no debería ser demasiado compleja ya que utilizaremos un archivo de tipo **MSI** que nos guiará a lo largo de cada uno de los pasos.

El primer paso es descargarnos el instalable del siguiente enlace: https://www.mongodb.com/try/download/community?tck=docs_server

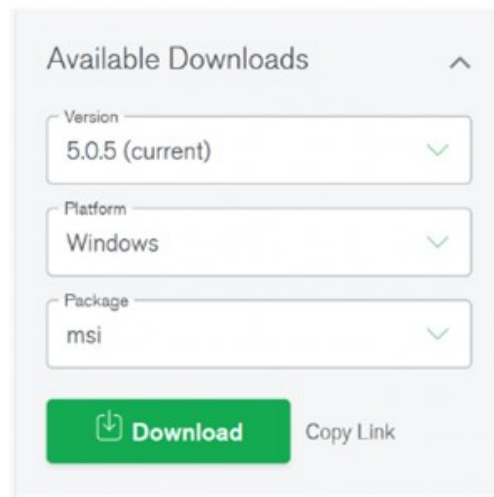


Figura 32. Descarga recomendada para Windows. Fuente: elaboración propia.

Durante la instalación se nos requerirá que elijamos la forma de instalar. En nuestro caso, para empezar a trabajar, podemos hacerlo de manera **Completa**.

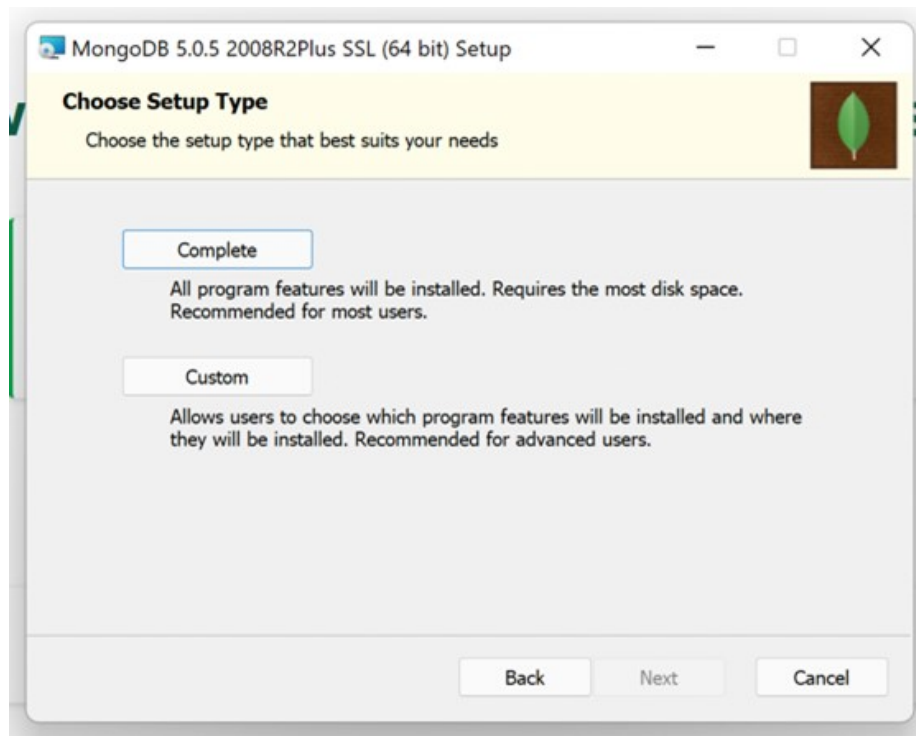


Figura 33. Instalación en sistema operativo Windows. Fuente: elaboración propia.

En la siguiente ventana se nos plantea la posibilidad de instalar MongoDB como un servicio. Esto implica que vamos a tener el servidor de bases de datos constantemente arrancado dentro de nuestro ordenador. Vamos a dejarlo activado ya que más adelante veremos cómo podemos acceder al servicio generado.

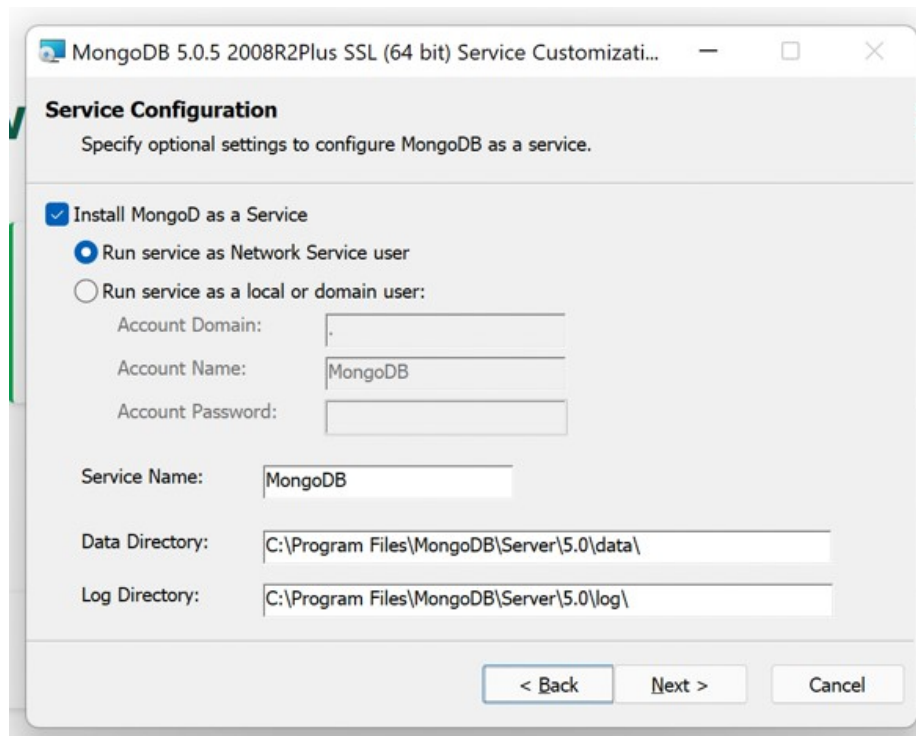


Figura 34. Creación del servicio dentro de Windows. Fuente: elaboración propia.

En la siguiente ventana dejamos activa la instalación de MongoDB Compass como aplicación para la gestión de bases de datos MongoDB.

Una vez termine la instalación dentro de nuestra máquina podemos comprobar si hemos realizado la instalación correctamente abriendo la aplicación **Services** que tenemos disponible en todos los sistemas Windows.

Tema 8. Mongo Setup

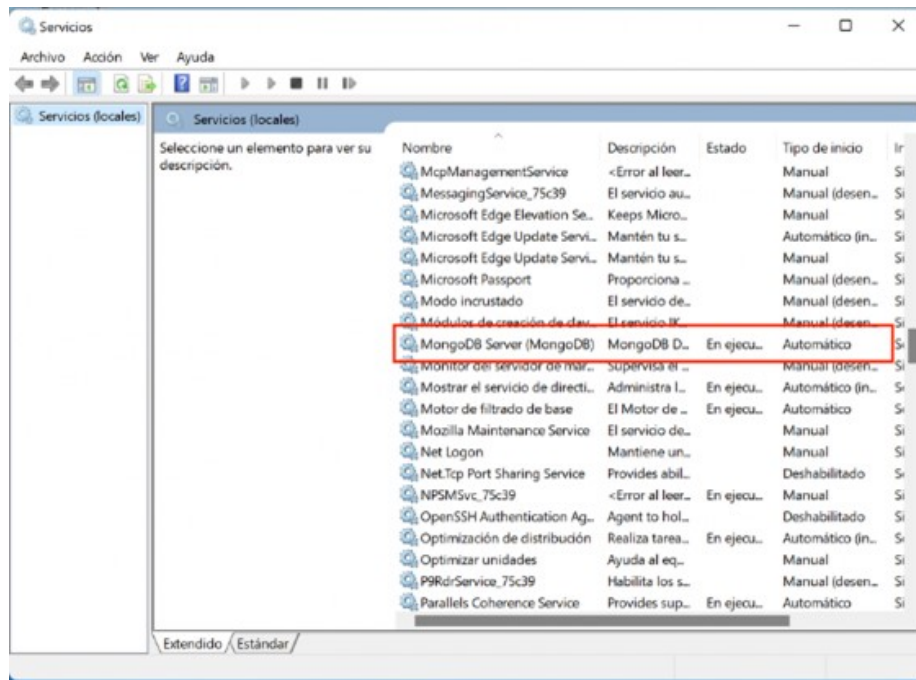


Figura 35. Comprobación de la ejecución del servicio de MongoDB. Fuente: elaboración propia.

Entre todos los servicios que tenemos instalados, deberíamos encontrar el que se llama **MongoDB Server**. Además, en el estado de este podemos ver cómo sale el valor **En ejecución**. Esta configuración nos indica que el servidor de MongoDB que hemos instalado se encuentra levantado y lo podemos usar.

El último paso sería tener acceso a los diferentes comandos de Mongo a nivel global dentro de nuestra máquina. Para ello vamos a agregar dentro de las variables de entorno la ruta donde se encuentran los ejecutables correspondientes.

Se puede comprobar que la ruta de instalación es la siguiente: C:\Program Files\MongoDB\Server\5.0\bin

(Dependiendo de la máquina puede cambiar la ruta, así que es recomendable comprobar si dentro de ese directorio encontramos el ejecutable **mongo.exe**). El siguiente paso sería abrir las propiedades del sistema de nuestro ordenador.

Tema 8. Mongo Setup

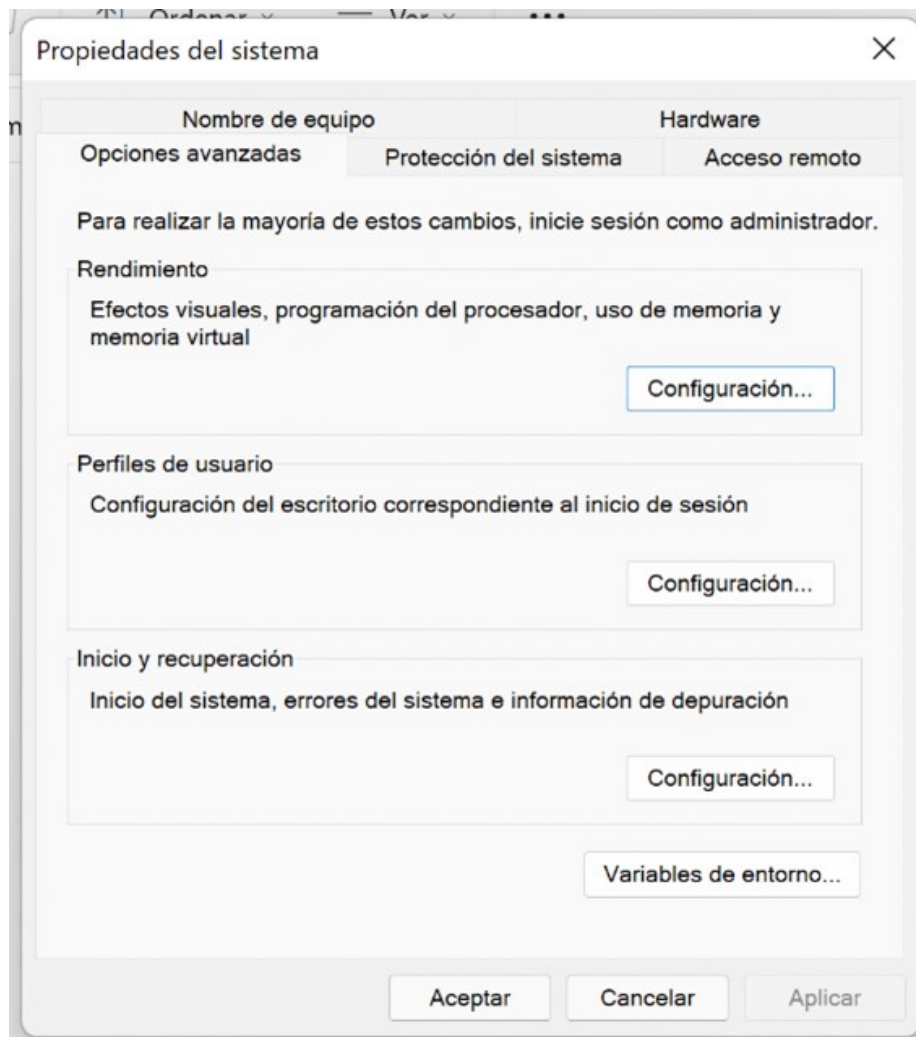


Figura 36. Propiedades del sistema Windows. Fuente: elaboración propia.

En el último botón pone **Variables de entorno...**

Tema 8. Mongo Setup

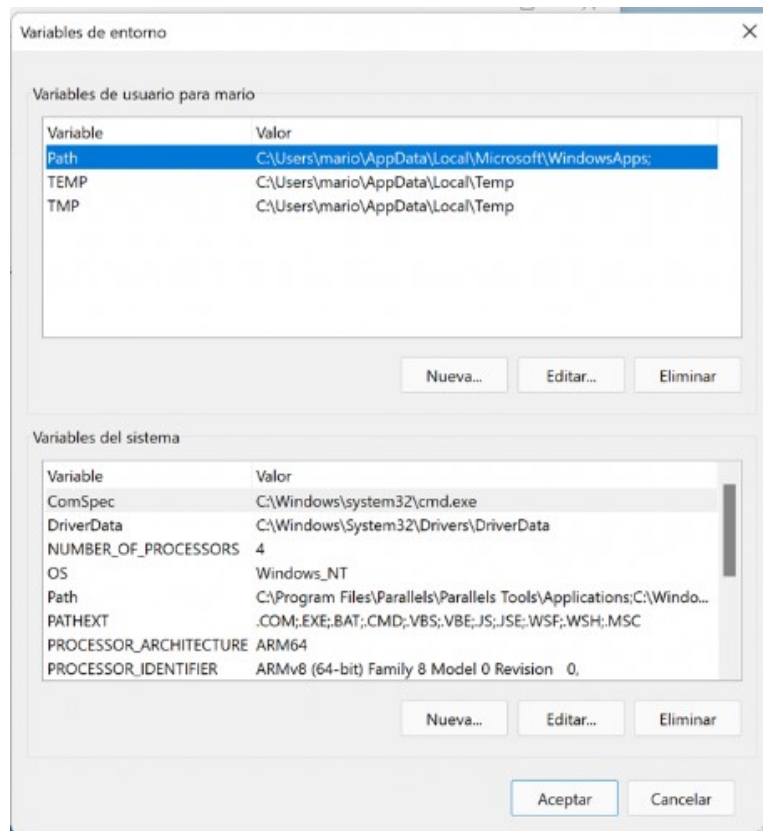


Figura 37. Modificación de variables de entorno. Fuente: elaboración propia.

En la parte de **Variables del sistema**, buscamos la que se llama **Path** y hacemos doble click sobre ella.

El último paso sería agregar dentro del Path la ruta donde hemos instalado MongoDB.

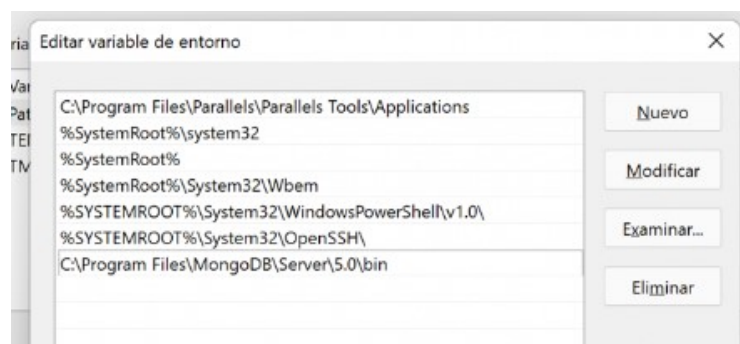
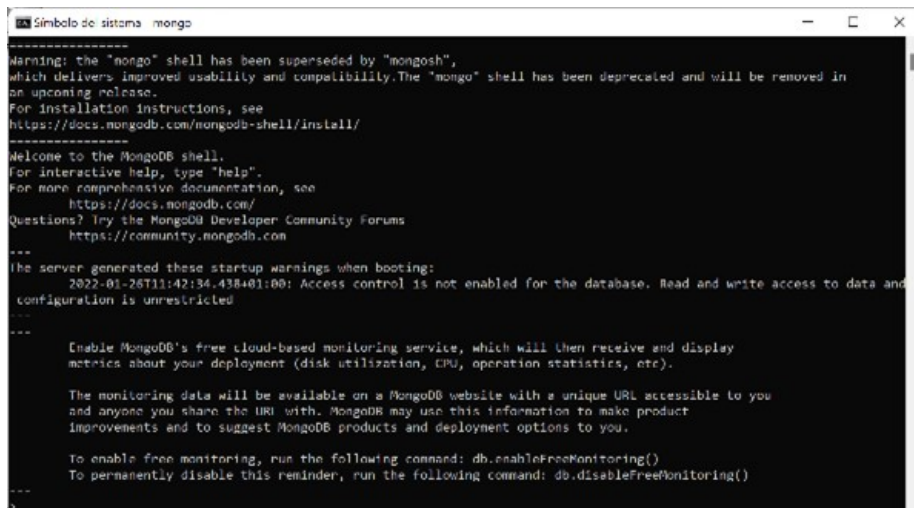


Figura 38. Rutas definidas dentro de la variable Path. Fuente: elaboración propia.

Tema 8. Mongo Setup

De esta manera, si abrimos un terminal dentro de nuestro ordenador, tecleamos el comando **mongo** y recibimos el siguiente mensaje quiere decir que todo está correctamente instalado.



```
Simbolo de sistema mongo
-----
Warning: the "mongo" shell has been superseded by "mongosh",
which delivers improved usability and compability. The "mongo" shell has been deprecated and will be removed in
an upcoming release.
For installation instructions, see
https://docs.mongodb.com/mongodb-shell/install/
-----
Welcome to the MongoDB shell.
For interactive help, type "help".
For more comprehensive documentation, see
https://docs.mongodb.com/
Questions? Try the MongoDB Developer Community Forums
https://community.mongodb.com
---
The server generated these startup warnings when booting:
  2022-01-26T11:42:34.438+01:00: Access control is not enabled for the database. Read and write access to data and
  configuration is unrestricted
---
  Enable MongoDB's free cloud-based monitoring service, which will then receive and display
  metrics about your deployment (disk utilization, CPU, operation statistics, etc).

  The monitoring data will be available on a MongoDB website with a unique URI accessible to you
  and anyone you share the URI with. MongoDB may use this information to make product
  improvements and to suggest MongoDB products and deployment options to you.

  To enable free monitoring, run the following command: db.enableFreeMonitoring()
  To permanently disable this reminder, run the following command: db.disableFreeMonitoring()
---
>
```

Figura 39. Comprobación del funcionamiento de MongoDB dentro del terminal. Fuente: elaboración propia.

Nuestro trabajo más adelante será interactuar con esta consola o generando scripts para que sean ejecutados desde la misma.

8.3. Instalación de MongoDB para sistemas Linux

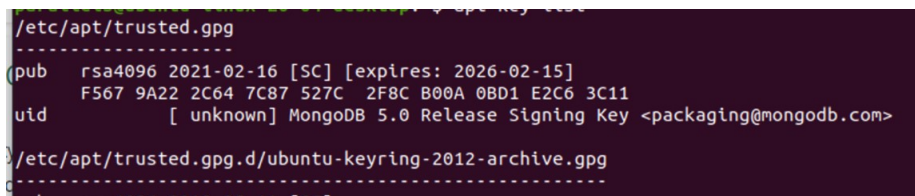
Para poder instalar MongoDB lo haremos de manera más sencilla siguiendo una serie de comandos a través del terminal.

El primer paso es importar la clave pública sobre la última versión estable de MongoDB. Lo podemos hacer lanzando el siguiente comando en el terminal:

```
curl -fsSl https://www.mongodb.org/static/pgp/server-5.0.asc | sudo apt-key add-
```

Este comando agrega la clave de MongoDB a nuestra lista de claves de confianza. La respuesta de este comando debería ser **OK**.

Podemos comprobar si la clave se ha agregado correctamente con el comando `apt-key list`.



```
/etc/apt/trusted.gpg
-----
pub   rsa4096 2021-02-16 [SC] [expires: 2026-02-15]
      F567 9A22 2C64 7C87 527C  2F8C B00A 0BD1 E2C6 3C11
uid   [ unknown] MongoDB 5.0 Release Signing Key <packaging@mongodb.com>
/etc/apt/trusted.gpg.d/ubuntu-keyring-2012-archive.gpg
-----
pub   rsa4096 2012-04-11 [SC] [expires: 2017-04-11]
      7798 14F6 3DD8 9B7D 3172  3844 4786 111D 23F1 4362
uid   [ unknown] Ubuntu Keyring (gpg)
-----
```

Figura 40. Comprobación del correcto funcionamiento de la clave. Fuente: elaboración propia.

Mediante el siguiente comando vamos a determinar el sitio donde APT debe ir a buscar las fuentes disponibles para posteriormente descargarlas.

```
echo "deb [ arch=amd64,arm64 ] https://repo.mongodb.org/apt/ubuntu
focal/mongodb-org/5.0 multiverse" | sudo tee
/etc/apt/sources.list.d/mongodb-org-5.0.list
```

Después de lanzar este comando actualizamos el índice del paquete local para que APT sepa dónde debe buscar la instalación.

```
sudo apt update
```

Tema 8. Mongo Setup

A partir de ahí ya podemos instalar MongoDB.

```
sudo apt install mongodb-org
```

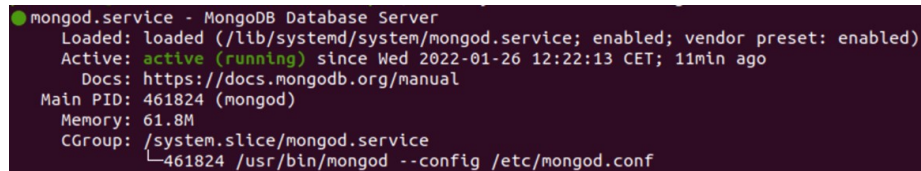
Una vez se instale el servidor podemos dejarlo arrancado como un servicio dentro de nuestra máquina. Siempre que necesitemos ponerlo en marcha podemos lanzar el siguiente comando.

```
sudo systemctl start mongod.service
```

Si necesitamos confirmar su estado, lo podemos hacer a través del siguiente comando.

```
sudo systemctl status mongod
```

Obteniendo este resultado en caso de estar arrancado:



```
● mongod.service - MongoDB Database Server
   Loaded: loaded (/lib/systemd/system/mongod.service; enabled; vendor preset: enabled)
   Active: active (running) since Wed 2022-01-26 12:22:13 CET; 11min ago
     Docs: https://docs.mongodb.org/manual
   Main PID: 461824 (mongod)
    Memory: 61.8M
   CGroup: /system.slice/mongod.service
           └─461824 /usr/bin/mongod --config /etc/mongod.conf
```

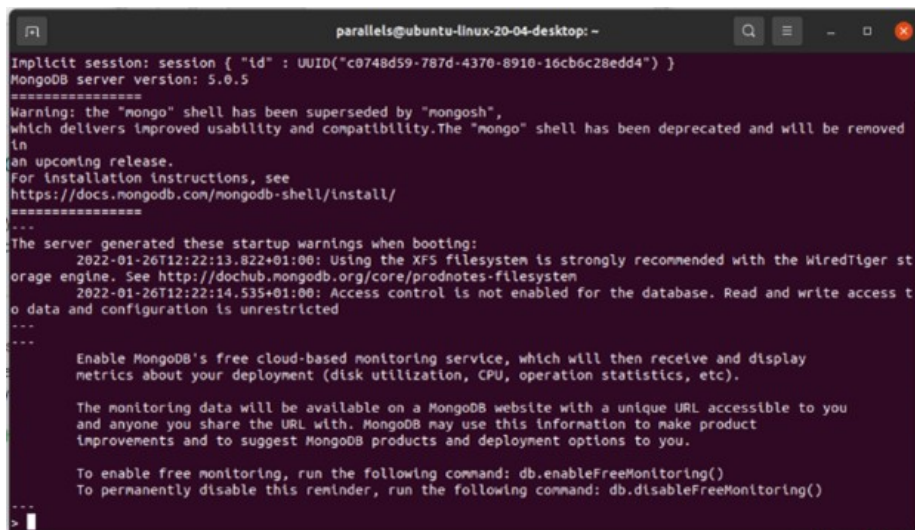
Figura 41. Comprobación del funcionamiento del servicio de MongoDB. Fuente: elaboración propia.

Si queremos que el servicio se inicie en el arranque de nuestra máquina, podemos lanzar el siguiente comando.

```
sudo systemctl enable mongod
```

A partir de aquí podemos comprobar su correcto funcionamiento lanzando el comando mongo sobre el terminal.

Tema 8. Mongo Setup

A terminal window titled 'parallels@ubuntu-linux-20-04-desktop: ~' showing the output of the 'mongo' command. The output includes session information, a warning about the 'mongo' shell being superseded by 'mongosh', startup warnings from the MongoDB server, and information about the free cloud-based monitoring service.

```
parallels@ubuntu-linux-20-04-desktop: ~  
Implicit session: session { "id" : UUID("c0748d59-787d-4370-8910-16cb6c28edd4") }  
MongoDB server version: 5.0.5  
-----  
Warning: the "mongo" shell has been superseded by "mongosh",  
which delivers improved usability and compatibility. The "mongo" shell has been deprecated and will be removed  
in an upcoming release.  
For installation instructions, see  
https://docs.mongodb.com/mongodb-shell/install/  
-----  
The server generated these startup warnings when booting:  
2022-01-26T12:22:13.822+01:00: Using the XFS filesystem is strongly recommended with the WiredTiger storage engine. See http://dochub.mongodb.org/core/prodnotes-filesystem  
2022-01-26T12:22:14.535+01:00: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted  
-----  
Enable MongoDB's free cloud-based monitoring service, which will then receive and display metrics about your deployment (disk utilization, CPU, operation statistics, etc).  
  
The monitoring data will be available on a MongoDB website with a unique URL accessible to you and anyone you share the URL with. MongoDB may use this information to make product improvements and to suggest MongoDB products and deployment options to you.  
  
To enable free monitoring, run the following command: db.enableFreeMonitoring()  
To permanently disable this reminder, run the following command: db.disableFreeMonitoring()  
-----  
>
```

Figura 42. Comprobación del funcionamiento del comando mongo dentro del terminal. Fuente: elaboración propia.

Si obtenemos el resultado de arriba quiere decir que ya tenemos todo lo necesario para empezar a trabajar con MongoDB.

8.4. Instalación de MongoDB para sistemas MacOS

Para la instalación de MongoDB sobre nuestros sistemas MacOS vamos a utilizar la herramienta **BREW**, la cual nos facilita el trabajo en la instalación de multitud de aplicaciones.

Si no la tenemos instalada podemos hacerlo a través del siguiente comando en el terminal:

```
/bin/bash -c "$(curl -fsSL <a href="https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh" target="_blank">https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh</a>)"
```

Una vez instalado ejecutamos el comando de actualización:

```
brew tap mongodb/brew
```

Cuando termine podemos lanzar la instalación del servidor de MongoDB.

```
brew install<strong> </strong><a href="mailto:mongodb-community@5.0" target="_blank">mongodb-community@5.0</a>
```

La instalación quizá tarde un poco pero no debería requerir ningún tipo de interacción por nuestra parte.

Podemos levantar el servicio de MongoDB con el siguiente comando:

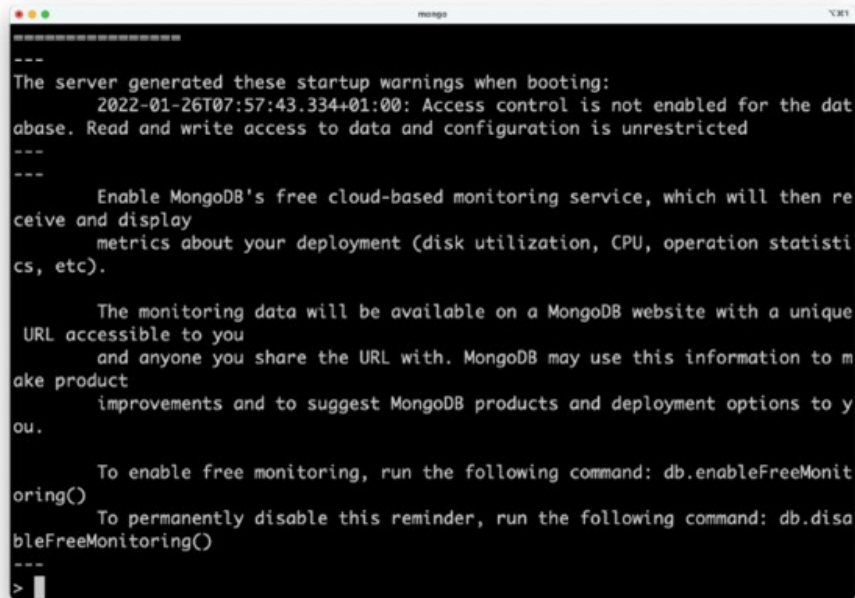
```
brew services start<strong> </strong><a href="mailto:mongodb-community@5.0" target="_blank">mongodb-community@5.0</a>
```

De esta manera tendremos el servidor siempre disponible cuando necesitemos trabajar con él. En caso de necesitar parar este servicio podemos lanzar el mismo comando, pero en vez de start, sustituirlo por stop.

Para comprobar que todo va según lo esperado podemos lanzar el comando **mongo**

Tema 8. Mongo Setup

dentro del terminal.



```
---
The server generated these startup warnings when booting:
  2022-01-26T07:57:43.334+01:00: Access control is not enabled for the dat
abase. Read and write access to data and configuration is unrestricted
---
---
  Enable MongoDB's free cloud-based monitoring service, which will then re
ceive and display
  metrics about your deployment (disk utilization, CPU, operation statisti
cs, etc).

  The monitoring data will be available on a MongoDB website with a unique
URL accessible to you
  and anyone you share the URL with. MongoDB may use this information to m
ake product
  improvements and to suggest MongoDB products and deployment options to y
ou.

  To enable free monitoring, run the following command: db.enableFreeMonit
oring()
  To permanently disable this reminder, run the following command: db.disa
bleFreeMonitoring()
---
> |
```

Figura 43. Comprobación del funcionamiento del comando mongo dentro del terminal. Fuente: elaboración propia

Si obtenemos este resultado quiere decir que se ha configurado todo perfectamente y estamos preparados para poder utilizar bases de datos MongoDB dentro de nuestro sistema.

Tema 9. Mongo Shell y JSON

9.1. Introducción y objetivos

Según vayamos conociendo las tecnologías para la creación del back de nuestras aplicaciones iremos incluyendo diferentes librerías que nos van a ayudar en el trabajo de recuperar los datos de nuestras bases de datos.

Antes de llegar a ese punto es imprescindible conocer cómo funciona la base de datos al más bajo nivel, qué posibilidades nos ofrece y cómo le podemos sacar partido de aquí en adelante.

Las bases de datos MongoDB no se parecen mucho a las bases de datos relacionales más tradicionales y es por eso por lo que este repaso que vamos a realizar a las entrañas de la base de datos a partir de este tema es imprescindible para poder alcanzar objetivos más complejos.

Hay que tener en cuenta que, en muchas ocasiones, vamos a salir de nuestra zona de confort centrada en el editor de código y estaremos trabajando sobre el terminal, pero es algo pasajero y con el objetivo de aprender nuevas técnicas y formas de interacción con datos.

Tema 9. Mongo Shell y JSON

9.2. Configuración inicial de Mongo Shell

En nuestro camino hasta conocer todos los entresijos de MongoDB vamos a comenzar analizando en qué consiste la **shell** propiamente dicha.

Se trata de realizar cualquier tipo de acción sobre nuestras bases de datos directamente con las herramientas que nos proporciona MongoDB y en este caso concreto a través del propio terminal nativo de la base de datos. Mediante la **shell de MongoDB** vamos a poder llevar a cabo cualquier interacción sobre los datos, así como diferentes tareas administrativas.

Si ya tenemos el servidor de MongoDB levantado sobre el puerto por defecto (27017) únicamente tenemos que arrancar la Shell a través del comando **mongo**.

En este momento ya estaríamos dentro del servidor de bases de datos y podemos empezar a lanzar comandos:

```
> db
```

Nos devuelve el nombre de la base de datos con el que estamos trabajando. Por defecto debería ser **test**.

Si queremos generar otra base de datos totalmente diferente lo podemos hacer a partir del comando:

```
> use unir-db
```

Genera la nueva base de datos y nos cambia a la misma para poder empezar a usarla.

Si necesitamos recuperar todas las bases de datos generadas en el servidor donde estamos trabajando podemos hacerlo con:

Tema 9. Mongo Shell y JSON

```
> show dbs
```

Si nuestra base de datos no dispone de ningún documento insertado no aparecerá en la lista.

A partir de la instancia **db**, como ya veremos más adelante en detalle, podemos insertar elementos o recuperarlos de manera muy sencilla:

```
> db.miColeccion.insertOne({ test: 'valor'})
```

Con este comando y con la base de datos recién iniciada estamos llevando a cabo varias acciones:

- ▶ Estamos generando la propia base de datos, que no existía hasta el momento.
- ▶ Creamos de igual manera la colección, llamada **miColeccion** dentro de la base de datos que estemos usando.
- ▶ Dentro de esta nueva colección incluimos un documento con una clave y un valor.

Si queremos recuperarlo, de igual manera podemos lanzar lo siguiente:

```
> db.miColeccion.find( )
```

Esta sentencia nos va a devolver tantos documentos como hayamos insertado dentro de esta colección y cada uno identificado con un ID diferente.

Si queremos que la visualización de los datos sea más amigable podemos usar el método **pretty**:

```
> db.miColeccion.find().pretty()
```

Podemos escribir acciones más complejas dentro del terminal de MongoDB. Incluso acciones multi línea para las cuales necesitamos ocupar una serie de espacios más amplios que los que utilizamos para los comandos habituales.

Tema 9. Mongo Shell y JSON

Podríamos generar variables o incluso condicionales:

```
> lex x = 5
> if (x < 10){
... x++;
... print(x);
}
```

Hasta que MongoDB no detecta el carácter encargado de cerrar el ámbito (en este caso las llaves del IF) no termina con el comando en curso.

Otras características que también están presentes dentro de la consola de MongoDB son las siguientes:

- ▶ Mediante las flechas arriba/abajo podemos recuperar el historial de comandos lanzados desde la terminal.
- ▶ Para autocompletar cualquier acción podemos usar la tecla de tabulador. Nos aporta cierta velocidad cuando ya conocemos los métodos que vamos a ejecutar.

Si necesitamos terminar nuestra sesión dentro de la Shell de MongoDB simplemente debemos lanzar el comando **exit**.

Personalizando la Shell

La escritura de sentencias y acciones dentro de una interfaz sobre línea de comandos como la que estamos analizando no es de lo más cómodo que nos vamos a encontrar. Existen algunas técnicas que nos permiten personalizar algunos aspectos cuando trabajamos directamente en la shell.

Mediante la modificación de la variable prompt podemos cambiar cómo se muestra el comienzo de la línea antes de lanzar cualquier tipo de comando. Esta variable puede incluir cualquier tipo de código JavaScript.

Tema 9. Mongo Shell y JSON

Por ejemplo, si queremos que nos marque cuántas acciones llevamos realizadas dentro de una sesión en concreto podemos proceder de la siguiente manera dentro del propio terminal:

```
> cmdCount = 1
> prompt = function ( ) {
... return (cmdCount++) + "> ";
... }
```

De esta manera se irán contando todas las acciones dentro de una misma sesión.

Si, por ejemplo, quisiésemos representar el nombre de la base de datos y el host con el que estamos trabajando, lo podríamos hacer a partir de las siguientes sentencias:

```
> host = db.serverStatus().host;
> prompt = function ( ) {
... return db + "@" + host + "$ ";
... }
```

De esta manera siempre tendremos una referencia en pantalla sobre la base de datos con la que estamos trabajando y evitaremos problemas.

Otro ejemplo podría mostrar el tiempo que lleva el sistema levantado y el número de documentos que tenemos almacenados:

```
> prompt = function (){
... return "Uptime:" + db.serverStatus().uptime + " Documents:" +
db.stats().objects + " > ";
}
```

Como casi todas las herramientas por la línea de comandos, disponemos de la posibilidad de lanzar **mongo --help** sobre el terminal para poder recuperar todas las ayudas que nos ofrece esta interfaz.

Tema 9. Mongo Shell y JSON

9.3. Creación de scripts para la Shell

Podemos definir ciertos ficheros Javascript que se encarguen de realizar acciones sobre nuestra base de datos, ya sea sobre los propios documentos o ciertas labores administrativas.

La parte positiva del uso de estos pequeños scripts es que podemos sacar el desarrollo a un editor mucho más amable que el que nos ofrece el propio terminal.

Podemos usar muchas de las sentencias que usamos habitualmente en Javascript, pero no tenemos disponibles todas las herramientas que nos aporta el lenguaje. Trabajaremos con unas acciones un tanto limitadas.

Podemos generar un fichero **pruebaConexion.js** con el siguiente contenido:

```
conn = new Mongo();  
db = conn.getDB("unir-test");
```

Mediante la utilización del objeto Mongo estamos creando una nueva conexión contra el servidor de MongoDB activo dentro de nuestra máquina.

Posteriormente recuperamos la base de datos con la que vamos a interactuar a través del método **getDB**. A partir de estas sentencias ya tendríamos disponible la base de datos para poder lanzar cualquier acción.

Para ejecutar este script únicamente hay que ejecutar:

```
mongo pruebaConexion.js
```

Por ejemplo, si queremos recuperar los datos de una colección en concreto, podemos completar nuestro script de la siguiente manera:

```
conn = new Mongo();  
db = conn.getDB("unir-test");
```

Tema 9. Mongo Shell y JSON

```
cursor = db.people.find();
while (cursor.hasNext()) {
  printjson(cursor.next());
}
```

Más adelante veremos con más profundidad los diferentes métodos que nos permiten interactuar con los documentos contenidos en las diferentes colecciones.

A partir de este tipo de ficheros podremos ir generando acciones sin necesidad de estar creando este tipo de interacciones dentro de la shell, que siempre es más incómodo.

Tema 9. Mongo Shell y JSON

9.4. Tipos de datos

El tipo de documentos BSON dispone de una serie de datos más específicos que los que nos proporcionan los ficheros de tipo JSON. Es importante conocer esta serie de tipos para poder aplicarlos en nuestros diferentes desarrollos.

Vamos a analizar únicamente aquellos tipos que son diferentes a los tipos habituales de los ficheros JSON.

Date

Dentro de la Shell de MongoDB podemos recuperar la fecha a partir de las siguientes sentencias:

- ▶ **Date ()** – devuelve la fecha actual en formato string.
- ▶ **new Date ()** – devuelve un objeto con la fecha en formato internacional.
- ▶ **ISODate ()** – devuelve un objeto igual que el anterior objeto.

```
date = new Date();  
print(date);
```

ObjectId

La shell de MongoDB nos ofrece la posibilidad de generar objetos de tipo ObjectId para poder definir los identificadores únicos para cada uno de los documentos que generemos dentro de nuestra base de datos.

```
conn = new Mongo();  
db = conn.getDB("unir-test");  
print(new ObjectId);
```

Tema 9. Mongo Shell y JSON

La creación de estos identificadores se hace de manera aleatoria utilizando la fecha de creación y ciertos contadores que se inicializan de manera aleatoria para cada una de las colecciones.

NumberLong

Por defecto, todos los números dentro de la base de datos de MongoDB se tratan como números con coma flotante. Para todos aquellos números que superan los 64 bits de tamaño disponemos del tipo NumberLong.

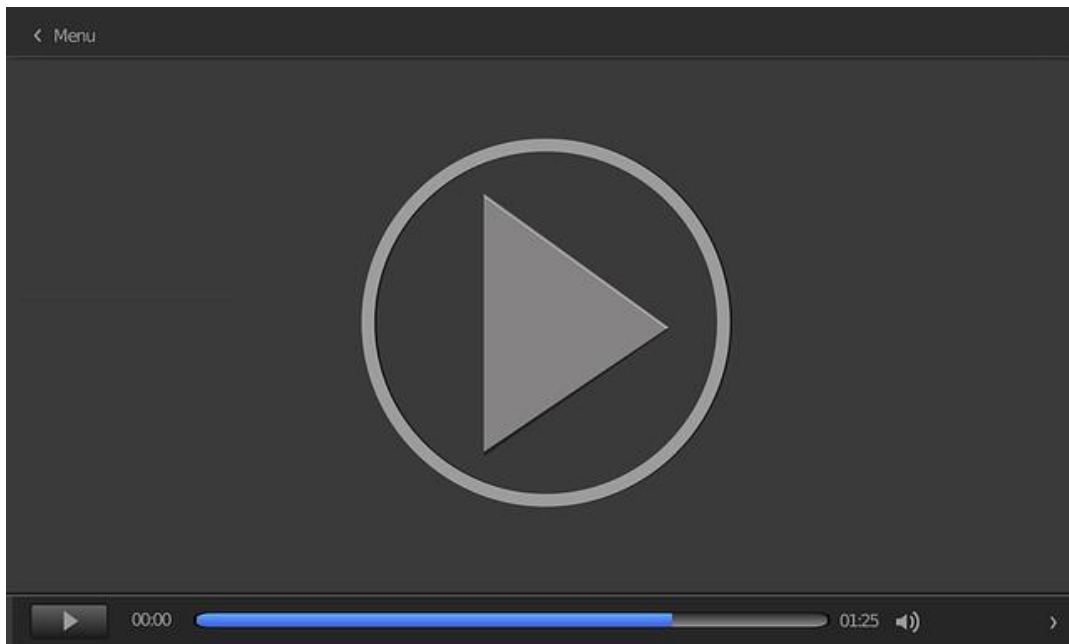
Se pueden crear este tipo de números a través de strings:

```
print(NumberLong("2090845886852"));
```

Para números más bajos podemos trabajar con **NumberInt** y **NumberDecimal**.

En el siguiente vídeo, *Introducción al manejo de la Shell para MongoDB*, se describen las pautas iniciales para el correcto uso de la Shell de MongoDB y sus acciones principales en el momento de crear y gestionar nuestras bases de datos.

Tema 9. Mongo Shell y JSON



Accede al vídeo:

<https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=208d09e0-9ed6-41f0-b55c-ae4d0140c1c8>

Tema 10. CRUD y Query/Update operators

10.1. Introducción y objetivos

Quizá uno de los posibles bloqueos que tenemos los desarrolladores web cuando nos enfrentamos al manejo de bases de datos tradicionales es la necesidad de aprender un nuevo lenguaje y herramientas totalmente diferentes a las que estamos acostumbrados.

En el manejo de bases de datos con MongoDB no vamos a tener esta serie de problemas ya que el manejo de la base de datos directamente desde la propia base de datos se hace a través de objetos, propiedades y métodos, características que estamos acostumbrados a usar en nuestro día a día.

Podríamos equiparar este aprendizaje al que tenemos cuando nos enfrentamos a una librería de terceros. Debemos saber cómo, con las herramientas sintácticas básicas que ya conocemos, podemos realizar acciones concretas que cumplan nuestros objetivos.

En este tema haremos un resumen de los métodos más básicos para realizar diferentes interacciones contra los datos introducidos dentro de una base de datos MongoDB y que posteriormente nos servirán para poderlos usar dentro de nuestras aplicaciones de back.

Tema 10. CRUD y Query/Update operators

10.2. Concepto de CRUD

Dentro del mundo del desarrollo web, las siglas CRUD nos van a acompañar en múltiples ámbitos, ya sea diseñando una base de datos o desarrollando una aplicación compleja para Front o Back.

En el ámbito de la creación de APIs es un concepto muy importante ya que nos permite aislar las acciones más importantes que vamos a realizar sobre nuestros datos. De alguna manera nos va a permitir generar las tareas más básicas de manejo de datos para poder ir creciendo en función de esa base.

Más adelante, cuando empecemos a desarrollar aplicaciones, veremos cómo vamos a tener que implementar casi todas estas acciones para todos los modelos que interactúen con una base de datos, convirtiéndose así en uno de los trabajos más habituales.

Las siglas CRUD las podemos explicar de la siguiente manera:

Create

Se trata de aquella función o conjunto de funciones que se encargan de generar un recurso para el modelo con el que estamos trabajando. Si hablásemos de nuestra tabla o colección empleados, la acción de Create se encargaría de recibir los datos necesarios para la creación de un empleado y persistirlo dentro de la base de datos.

Además, esta acción suele ser la encargada de definir el identificador único para cada uno de los recursos que insertemos.

Read

Tema 10. CRUD y Query/Update operators

Conjunto de acciones que nos permite recuperar la totalidad de los recursos de una colección o incluso un conjunto filtrado de los mismos. Dependiendo de los datos que vayamos a utilizar las funciones de lectura se especializan con unos filtros u otros.

Update

Se trata de la función encargada de editar uno o varios recursos dentro de nuestra base de datos. De manera habitual va a necesitar los nuevos datos para el registro y además el identificador único del mismo para poder llevar a cabo su labor.

Delete

Conjunto de funciones para lograr borrar elementos de nuestras bases de datos. Es importante que las acciones que son irreversibles siempre vayan filtradas sobre los elementos con los que vamos a interactuar para no eliminar información necesaria.

Tema 10. CRUD y Query/Update operators

10.3. Create Operations

Vamos a analizar, generando scripts para la Shell de MongoDB, cómo podemos realizar acciones para insertar documentos dentro de las diferentes colecciones con las que iremos trabajando.

Inserción de un único documento

Disponemos de la función **insertOne** para persistir un documento dentro de una colección en concreto.

```
db.products.insertOne({
  name: 'Frigorifico',
  stock: 50,
  department: 'home',
  available: true,
  tags: ['kitchen', 'basic', 'offer']
});
```

Mediante esta acción le estamos diciendo a nuestra base de datos que dentro de la colección products (si no existe, se crea) debemos insertar el documento que pasamos por parámetro.

Como resultado de esta acción obtenemos el nuevo documento creado con el identificador único que le asigna nuestro servidor de MongoDB (**_id**).

Inserción de múltiples documentos

Para la acción de insertar a la vez múltiples documentos sobre una colección disponemos del método **insertMany**.

```
db.products.insertMany([
  { name: 'Lavadora', stock: 20, department: 'home', available: true, tags:
```

Tema 10. CRUD y Query/Update operators

```
['kitchen', 'basic'] },  
  { name: 'Lápiz', stock: 250, department: 'office', available: true, tags:  
    ['basic'] },  
  { name: 'Camiseta azul', stock: 10, department: 'man', available: false,  
    tags: ['summer'] }  
]);
```

Para utilizar este método lo único que debemos cambiar es que por parámetro le pasamos un array con todos los datos de los documentos que vayamos a insertar dentro de la colección.

El resultado de esta acción nos devuelve un objeto con la lista de los ids asignados a cada uno de los nuevos documentos.

Características a tener en cuenta de la creación

Si la acción de creación de un documento se realiza sobre una colección que no existe, **esta se genera automáticamente dentro de la base de datos.**

La clave primaria para todos los documentos insertados es `_id`. Si en la inserción del documento omitimos dicha clave única, MongoDB se encarga de generarla por nosotros.

Las operaciones de escritura sobre un único documento son atómicas, es decir, o se realizan o nos devuelven el error asociado explicando el porqué de su fallo.

Tema 10. CRUD y Query/Update operators

10.4. Read Operations

Vamos a analizar las diferentes acciones que tenemos disponibles para recuperar documentos de las diferentes colecciones de nuestra base de datos.

Seleccionar todos los documentos de una colección

Disponemos del método **find** para poder realizar esta acción.

```
result = db.products.find({}).toArray();  
printjson(result);
```

(Transformamos el resultado a array y utilizamos el método printjson para poder visualizarlo de manera más cómoda).

Filtros sobre un valor

Si queremos recuperar una serie de documentos a través de un valor concreto podemos modificar el objeto que le pasamos como primer parámetro al método find.

```
result = db.products.find({ department: 'home' }).toArray();  
printjson(result);
```

De esta manera estamos generando un **filtro estricto** que nos devolverá todos aquellos elementos que coincidan con que su propiedad **department** es igual al valor **home**.

No debemos confundir este tipo de filtros con búsquedas específicas de palabras o sobre parte de una misma. En este caso debe cumplir exactamente con el filtro que hemos creado.

Tema 10. CRUD y Query/Update operators

Creando filtros a través de operadores

Si necesitamos realizar un tipo de filtro más específico sobre las propiedades de nuestros documentos podemos usar una serie de operadores disponibles dentro de MongoDB.

```
result = db.products.find(
{ tags: { $in: ['offer', 'basic'] } }
).toArray();
printjson(result);
```

En el ejemplo anterior, mediante el operador `$in` buscamos todos aquellos documentos cuya propiedad `tags` tenga alguno de los valores especificados a partir del operador.

Condiciones AND

Puede ser que necesitemos recuperar documentos que cumplan con varios filtros a la vez. Lo único que tenemos que hacer es especificar los diferentes valores dentro del objeto que pasamos como parámetro al método.

```
result = db.products.find(
{ available: true, department: 'home' }
).toArray();
printjson(result);
```

Para que la sentencia nos devuelva un documento, este tiene que cumplir de manera estricta con los dos valores que se especifican como filtro del `find`.

En caso de necesitar hacer algún tipo de filtro más complejo, podemos utilizar los diferentes operadores disponibles dentro de MongoDB:

- `$eq` – encuentra los documentos cuya propiedad filtrada sea igual a un valor concreto.

Tema 10. CRUD y Query/Update operators

- ▶ `$gt` – comprende todos aquellos documentos cuyo valor filtrado sea mayor que otro valor específico.
- ▶ `$gte` – lo mismo que el anterior, pero en este caso el valor puede ser mayor o igual.
- ▶ `$lt` – lo utilizamos para filtros sobre valores menores.
- ▶ `$lte` – lo mismo que el anterior, pero para el filtro menor o igual.
- ▶ `$in` – comprueba si el valor está contenido dentro de un array.
- ▶ `$ne` – comprueba si el valor no es igual a otro concreto.
- ▶ `$nin` – comprueba si el valor no se encuentra incluido dentro de un array.

Por ejemplo, si quisiese recuperar **todos los productos activos con un stock superior a 100 unidades**, podría utilizar lo siguiente:

```
result = db.products.find(
  { available: true, stock: { $gte: 100 } }
).toArray();
printjson(result);
```

Condiciones OR

Del mismo modo que las anteriores, tenemos la posibilidad de poder lanzar filtros en los que vamos a recuperar documentos que cumplan con alguna de las opciones planteadas.

```
result = db.products.find(
  { $or: [{ stock: { $gte: 100 } }, { department: 'home' }] }
).toArray();
printjson(result);
```

En el ejemplo, recuperamos todos aquellos productos que o tengan un stock superior a 100 unidades o se encuentren dentro del departamento home.

Tema 10. CRUD y Query/Update operators

Para conseguir este objetivo debemos lanzar el operador \$or y pasarle como valor un array con las diferentes condiciones que queremos llevar a cabo.

Se podrían combinar las condiciones AND y OR siempre y cuando sigamos las especificaciones de cada una de ellas.

Tema 10. CRUD y Query/Update operators

10.5. Update Operations

Disponemos de diferentes métodos para poder modificar los datos de nuestros documentos a través de filtros.

En este tipo de acciones, al igual que pasa en las bases de datos relacionales, hay que prestar especial atención al filtro con el que vamos a trabajar. Si el filtro falla o está mal definido podemos enfrentarnos a actualizaciones completas de colecciones de manera inesperada.

Para las acciones de actualización disponemos del operador `$set` que nos ayudará a la hora de concretar qué datos son los que vamos a modificar.

Actualización de un único documento

Para poder alcanzar este objetivo disponemos del método **updateOne** el cual modifica el primer documento dentro de una colección que cumpla con el filtro creado.

```
result = db.products.updateOne(
  { department: 'home' },
  {
    $set: { stock: 220 },
    $currentDate: { lastModified: true }
  }
)
printjson(result);
```

En este ejemplo recuperamos el primer documento que forme parte del departamento **home**. La modificación la realizamos sobre la propiedad `stock`.

Mediante el operador `$currentDate` le indicamos a la operación que debe modificar la propiedad `lastModified` (puede ser el nombre que queramos) del documento que

Tema 10. CRUD y Query/Update operators

estamos cambiando para que apunte la fecha de la modificación. Si la propiedad no existe dentro del documento se encarga de crearla.

Actualización de múltiples documentos

Para lograr actualizar varios documentos en la misma operación disponemos del método **updateMany**.

```
result = db.products.updateMany(
  { stock: { $lt: 50 } },
  {
    $push: { tags: 'offer' },
    $currentDate: { lastModified: true }
  }
)
printjson(result);
```

En este ejemplo estamos buscando todos aquellos productos que tengan un stock menor de 50 unidades y, en vez de modificarlos a través del operador \$set utilizamos el operador \$push que nos permite agregar un nuevo elemento dentro del array de tags.

Como en el caso anterior, esta operación también nos permite modificar la fecha de modificación del documento.

Reemplazando los datos de un documento

Mediante el uso del método **replaceOne** podemos cambiar el contenido completo de un documento específico. Los datos que pasemos como parámetro en la ejecución de dicho método serán las nuevas propiedades y valores del documento.

```
result = db.products.replaceOne(
  { name: 'Lavadora' },
  { name: 'Washing Machine', stock: 0, available: 0, department: 'home' }
)
```

Tema 10. CRUD y Query/Update operators

```
printjson(result);
```

Esta es una acción mucho más «peligrosa» ya que estamos situando una serie de propiedades nuevas en el lugar donde antes encontrábamos otras totalmente diferentes. Las propiedades del nuevo documento pueden ser incluso totalmente diferentes a las que tenía en un principio.

Tema 10. CRUD y Query/Update operators

10.6. Delete Operations

De todas del CRUD nos falta por analizar cómo podemos borrar documentos de nuestras diferentes bases de datos.

Borrar todos los documentos

Podemos borrar todos los documentos de una colección utilizando la función **deleteMany**.

(¡Cuidado! Si lanzamos esta prueba perderemos todos los documentos dentro de la colección products.).

```
result = db.products.deleteMany({});  
printjson(result);
```

El método nos devuelve un objeto indicándonos si la ejecución ha sido correcta y cuántos documentos se han visto afectados.

Borrar todos los documentos a partir de una condición

Al igual que para las actualizaciones, debemos concretar muy bien qué documentos necesitamos borrar para no encontrarnos con eliminaciones de datos no esperadas. El único cambio que debemos hacer con respecto al ejemplo anterior es el de especificar el filtro dentro de la función **deleteMany**.

```
result = db.products.deleteMany({  
  department: 'office'  
});  
printjson(result);
```

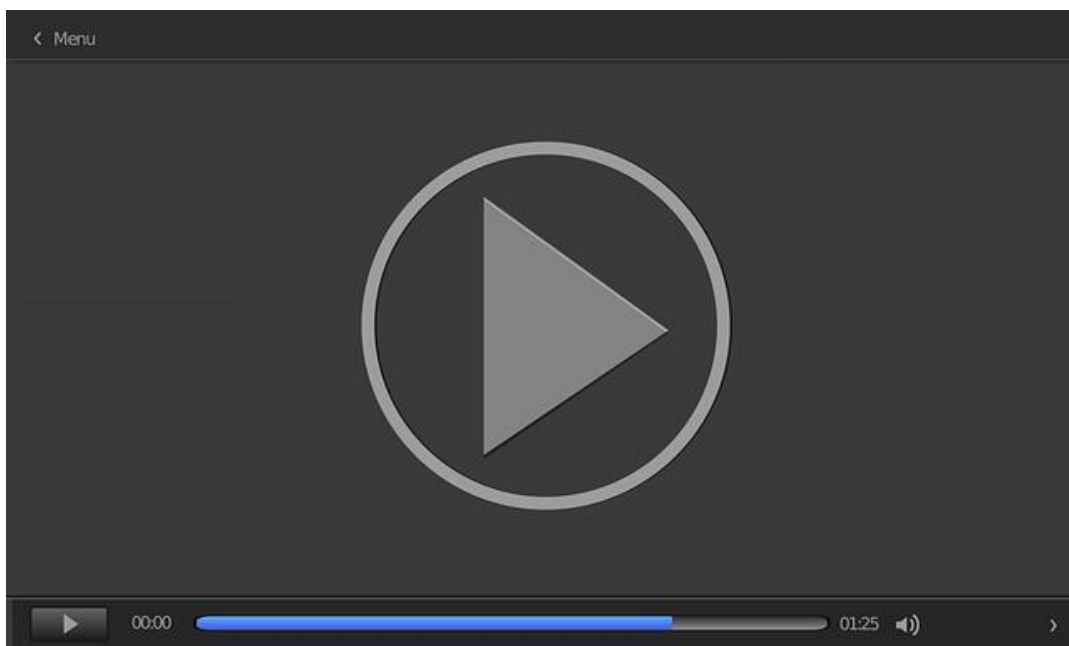
Tema 10. CRUD y Query/Update operators

Borrar un único documento a partir de una condición

Podemos utilizar el método **deleteOne** para eliminar definitivamente de una colección el primer documento que coincida con el filtro que pasemos al método.

```
result = db.products.deleteOne({
  stock: { $gte: 1000 }
});
printjson(result);
```

En este vídeo, *Creación y manejo de una colección*, analizaremos las acciones más importantes que podemos llevar a cabo sobre una colección en MongoDB, así como algunas sentencias de tipo find complejas.



Accede al vídeo:

<https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=30bac9a2-84e6-4589-b1f6-ae4d0147db0f>